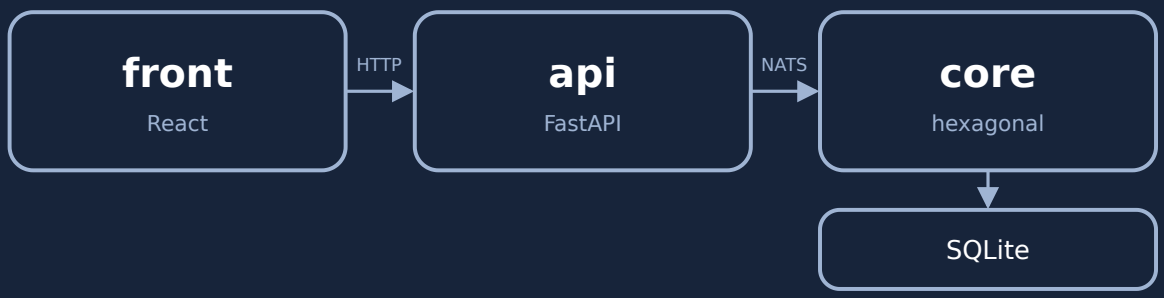
STUDY GUIDE

Prompt Vault, from the inside

Python, hexagonal architecture, NATS, FastAPI and React explained with a real project



Study material generated from the implemented and tested code of the three projects: core, api and front.

Contents

Click any entry to jump to its page.

0. Before you begin 5
- 0.1 How to study with this guide 5
- 0.2 Reading map 5
- 0.3 The colored boxes 6

1. The problem and the solution 7
- 1.1 Prompts, and why version them 7
- 1.2 What the system does 7
- 1.3 Three projects, one conversation 8
- 1.4 The map of the project 9
- 1.5 The technologies, in a table 10

2. Essential Python 12
- 2.1 The environment: uv, pyproject.toml and uv.lock 12
- 2.2 Modules, packages and imports 13
- 2.3 Types: saying what each thing is 13
- 2.4 Classes, dataclasses and enums 14
- 2.5 Protocol: contracts without inheritance 14
- 2.6 async / await: the waiter who serves several tables 15
- 2.7 Context managers: "with" and "async with" 16
- 2.8 Exceptions: when something cannot continue 16
- 2.9 A mini-dictionary of symbols 16

3. Hexagonal architecture 18
- 3.1 The idea: protect the rules from the technology 18
- 3.2 The five blocks 19
- 3.3 The Dependency Rule 19
- 3.4 Why it is worth it (and what it costs) 20
- 3.5 The patterns you will find 21

4. Domain: the heart 22
- 4.1 Rich modeling, not anemic 22
- 4.2 The aggregate: Prompt and its versions 22
- 4.3 Immutable versions and the active-version pointer 23
- 4.4 The life cycle: a state machine 24
- 4.5 Invariants: what is always true 25
- 4.6 Domain events: facts that happened 26
- 4.7 The diff and the Strategy pattern 26
- 4.8 A lesson the tests taught 28

5. Ports and Application 29
- 5.1 Ports: what the core demands from the world 29
- 5.2 Unit of Work: the "all or nothing" transaction 30
- 5.3 Use cases: the orchestra 30

5.4 Anatomy of a use case 31
5.5 Dependency injection and the power of the "fake" 32

6. Adapters and Infra 34
6.1 Infra: configuring the technologies 34
6.2 Persistence: SQLAlchemy and SQLite 34
6.3 Messaging: the NATS adapter 38
6.4 Structured logging: structlog + rich 42
6.5 main.py: the composition root 43

7. The HTTP API 45
7.1 The layers of the api 45
7.2 FastAPI in five minutes 47
7.3 The service builds the message 48
7.4 The publisher: the only door to NATS 49
7.5 Errors: one base class, one central place 50
7.6 The contract: Swagger and OpenAPI 53
7.7 Testing the api without a network 53

8. The front end 55
8.1 React in five minutes 55
8.2 The map of the front end 56
8.3 The service layer: Axios and the contract 56
8.4 State and routing in App 58
8.5 Loading data: why one request per card 59
8.6 The screens 59
8.7 Patterns that make it feel solid 62

9. A request end to end 64
9.1 The core's half of the story 64
9.2 The whole journey, from the browser 66
9.3 What each observer sees 67
9.4 When something goes wrong 67

10. Verifying that it works 69
10.1 Verify in layers 69
10.2 The smoke test 70
10.3 The global smoke test: do the three pieces agree? 70
10.4 Other verifications 71
10.5 Lessons from the way 71

11. Hands-on 73
11.1 Bringing up the environment 73
11.2 Recipes 73
11.3 Exercises 74
11.4 Natural next steps 75

A. Glossary 77

B. File map 79
B.1 core/ 79

B.2 api/ 80
B.3 front/ 81
B.4 repository root 81

C. Going further 82
C.1 Books and articles 82
C.2 Official documentation 82

D. Answers to the checkpoints 84

CHAPTER 0

0 Before you begin

Who this guide is for and how to study with it

Welcome to the class. This guide is written for someone who has never programmed in Python (or has done it very little) and wants to understand, for real, the **Prompt Vault** project: what every piece does, why it sits where it sits, and how the pieces talk to each other. Instead of explaining technologies in the abstract, we use the project itself as a laboratory. Every code listing you will see was copied straight from the real files by the build script of this guide, and every result of an execution was obtained by running the system.

The project has grown since the first edition of this guide. It used to be a single Python service that listened to messages. It is now **three projects that work together**:

Project	What it is	Language and main tools
`core/`	The business: prompts, versions, states, diff. It has no HTTP; it listens to NATS messages and stores data in SQLite.	Python 3.13, hexagonal architecture, NATS, SQLAlchemy
`api/`	An HTTP gateway. It validates requests, forwards each one to the core as a NATS message and turns the answer (or the error) into HTTP.	Python, FastAPI, Pydantic
`front/`	A browser dashboard to list, create, edit, compare, browse and recover prompts.	React, TypeScript, Vite, Tailwind CSS, Axios

0.1 How to study with this guide

1. **Read in order.** Every chapter opens with a guiding question and closes with a summary. Chapters 2 to 8 also contain a *checkpoint*: three questions to fix the ideas (the answers are in Appendix D).
2. **Open the files.** When the text cites a file, open it in your editor next to the guide. Seeing the real code fixes far more than reading about it.
3. **Run the commands of Chapter 11.** Sending a message and watching the system answer turns a concept into an experience.
4. **Do not try to memorize.** The goal is to know where to look and why each thing exists.
5. **Break things on purpose.** Change a rule, run a smoke test and see which check fails. Tests exist to be read as documentation of the behavior.

0.2 Reading map

Chapter	The question it answers
1 · The problem and the solution	What is the system for, and how does anyone talk to it?
2 · Essential Python	Which parts of the language do I need to read this code?

Chapter	The question it answers
3 · Hexagonal architecture	Why is the core split into five folders, and who may talk to whom?
4 · Domain	Where do the business rules live, and how do they protect themselves?
5 · Ports and Application	How does a request become a use case without depending on technology?
6 · Adapters and Infra	How does the core connect to NATS, SQLite and the terminal?
7 · The HTTP API	How do HTTP requests reach the core, and how do errors come back?
8 · The front end	How does the browser application turn all of this into screens?
9 · A request end to end	What happens, step by step, when someone clicks "Save" in the browser?
10 · Verifying that it works	How do we know it really works?
11 · Hands-on	How do I run, explore and extend the project myself?

■ 0.3 The colored boxes

CONCEPT

An idea or term that is new, explained slowly.

ANALOGY

A comparison with everyday life to anchor the idea.

IN THE PROJECT

Where that idea appears, concretely, in the Prompt Vault code.

WATCH OUT

A common trap or an honest limitation of the project.

LESSON LEARNED

Something that went wrong during development and what we learned from it.

CHECKPOINT

Three questions at the end of a chapter. The answers are in Appendix D.

CHAPTER 1

1 The problem and the solution

Why Prompt Vault exists and how the three projects talk to each other

> **GUIDING QUESTION**
>
> Why does this system exist, and how does a person use it, if the business core has no screen and no web address?

1.1 Prompts, and why version them

A **prompt** is the instruction text we send to a language model (an LLM, such as Claude): "summarize the text below in three bullet points, in a formal tone". Changing a single word can change the answer a lot. Anyone who works with LLMs ends up with dozens of prompts that evolve week after week, and uncomfortable questions appear: which version worked yesterday? who changed what? can I go back?

> **ANALOGY · The document history**
>
> Think of the version history of a Google Doc, or the commits of Git. Nobody erases the past: you only add new chapters and choose which one is "in force". Prompt Vault does exactly that, specialized in prompts.

1.2 What the system does

Capability	NATS subject	HTTP route (api)	Behavior
Create a prompt	`prompt.create`	`POST /prompts`	Born active, with version 1 already active.
Add a version	`prompt.version.create`	`POST /prompts/{id}/versions`	Appends a version; it is activated only if you ask (`activate`).
Change the content	`prompt.content.update`	`PUT /prompts/{id}/content`	Never edits: creates a new version and activates it.
Activate a version	`prompt.version.activate`	`POST /prompts/{id}/activate`	Chooses which version is "in force", including an old one (rollback).

Capability	NATS subject	HTTP route (api)	Behavior
Delete (soft delete)	`prompt.delete`	`DELETE /prompts/{id}`	Hides the prompt but keeps everything. It can be recovered.
Recover	`prompt.restore`	`POST /prompts/{id}/recover`	Returns to the state the prompt had before being deleted.
Read	`prompt.get`, `prompt.versions.list`, `prompt.list.active`, `prompt.list.deleted`	`GET /prompts/{id}`, `/versions`, `/active`, `/deleted`	Questions that only read, never change anything.
Compare versions	`prompt.versions.compare`	`POST /prompts/{id}/compare`	Shows lines added, removed and unchanged between two versions (a diff).

CONCEPT · The rules of the game

- Every change of content creates a new version. Nothing is overwritten.
- Only one version is active per prompt, at any moment.
- History is never lost. Deleting is "soft": the prompt disappears from the active lists but stays stored.
- These rules live in the heart of the system (the `domain` layer), not scattered around the code.

1.3 Three projects, one conversation

Look at the table of the previous section: every capability has **two doors**. The core listens to NATS subjects, and the api offers the same operations as HTTP routes. That is the central design decision of the whole system: the business core knows nothing about HTTP, browsers or web pages.

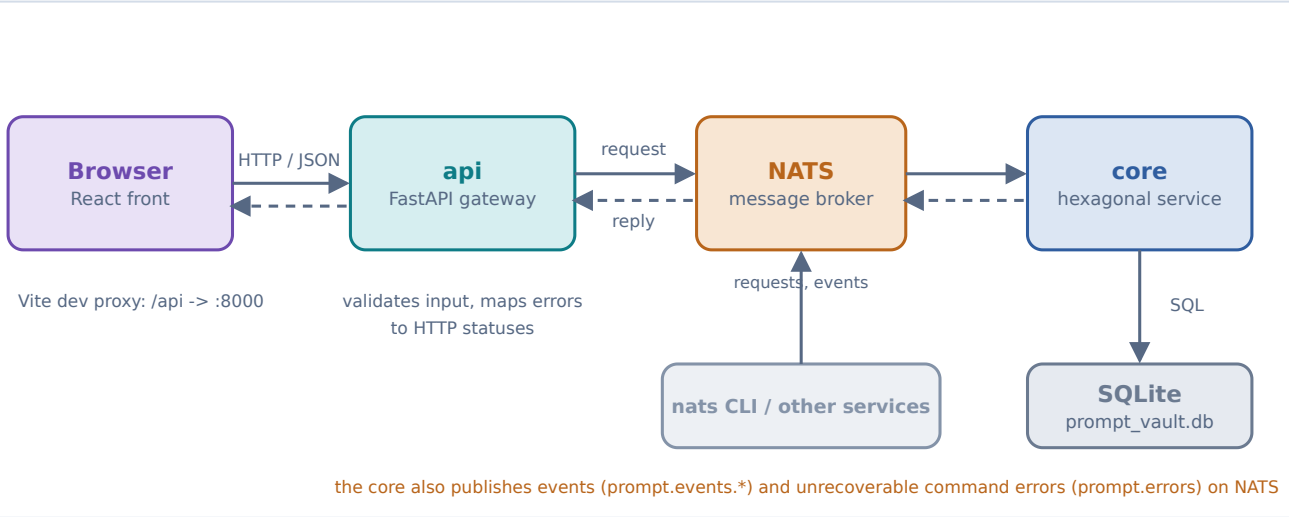


Figure 1 — The whole system: the browser talks HTTP to the api; the api talks NATS to the core; the core talks SQL to a file.

How do you read this figure? From left to right, a request crosses three borders. The **front** (running in a browser) sends HTTP/JSON to the **api**. The api does not hold any business rule: it turns the HTTP request into a **NATS message** and waits for the answer. The **core** receives the message, runs the use case, stores the result in **SQLite** and answers. Anything else that speaks NATS (the `nats` command-line tool, or a future service) can use the core directly, without going through the api.

> **ANALOGY · The radio dispatch center**
>
> Imagine a taxi radio dispatch center. You do not call a specific driver: you speak on a channel ("channel 7, I need a car") and whoever serves that channel answers. NATS is the center, the *subjects* are the channels, and the core is the crew that serves the channels `prompt.*`. The api is a receptionist: customers arrive at the counter (HTTP), and the receptionist talks to the radio on their behalf.

Here is a real conversation, made with the NATS CLI (a command-line tool to talk to NATS):

Terminal · real output of the system

```
$ nats request prompt.create '{"name":"cli","content":"hello"}' --raw
{"ok": true, "correlation_id": "25e14cde-cb7d-4f31-9fe5-29f8cce8cb65",
 "data": {"id": "f11c267d-...", "name": "cli", "status": "active",
          "active_version_id": "c763a3ff-...", "content": "hello", ...}}
```

The answer is a JSON **envelope**: `ok` says whether it worked, `correlation_id` is a tracking label (we will meet it in Chapter 6) and `data` carries the result, here the created prompt. The api uses exactly the same envelope, then returns only the `data` part to the browser.

■ 1.4 The map of the project

The project was built in phases. First the core, from the inside out (domain, then ports and application, the diff between versions and, finally, the adapters and infrastructure that connect everything to the real world). Then the HTTP api on top of it, and finally the browser front end. The repository looks like this:

Repository layout

```
prompt-vault/
├── README.md, smoke_test.sh      overview · global smoke test
├── docs/                         this guide and the interactive map
├── core/
│   ├── docker-compose.yml        local NATS server
│   ├── smoke_test.sh             smoke test through the NATS CLI
│   └── src/
│       ├── main.py               composition root: wires everything   (90 lines)
│       ├── domain/               pure business rules                  (605)
│       ├── port/                 contracts (Protocols)                 (57)
│       ├── app/                  use cases                            (316)
│       ├── adapter/
│       │   ├── persistence/      SQLAlchemy + SQLite                  (288)
│       │   └── messaging/        NATS                                 (589)
│       └── infra/                technology configuration             (150)
├── api/
│   ├── main.py                   FastAPI app, NATS lifespan            (32)
│   ├── controllers/              the HTTP router                      (115)
│   ├── services/                 PromptService                         (90)
│   ├── messaging/                NATSPublisher                        (108)
│   ├── models/                   DTOs and the response wrapper        (118)
│   ├── utils/                    exceptions, handlers, helpers        (195)
│   └── tests/                    117 unit tests                       (695)
└── front/
    ├── openapi.json              snapshot of the api contract
    └── src/                      React app, about 1,900 lines of TypeScript
```

1.5 The technologies, in a table

Technology	Version	What it is for here
Python	3.13	The language of the core and the api.
uv	0.13	Creates the environment, installs dependencies and runs programs.
NATS + nats-py	server 2.x · lib 2.16.0	The message bus: the "front door" of the core.
SQLite	built into Python	The database: a single file.
SQLAlchemy + aiosqlite	2.1.8 · 0.22.1	Translates Python objects into table rows, asynchronously.
structlog and rich	26.1.0 · 15.0.0	Structured logs, and readable colored terminal output.
FastAPI, Pydantic, uvicorn	0.162 · 2.14 · 0.54	The HTTP gateway: routes, validation and the server.
pytest + pytest-asyncio	9.2 · 1.4	The api unit tests.
React, Vite, TypeScript	19.3 · 8.3 · 5.9	The browser application and its tooling.
Tailwind CSS, Axios	4.3 · 1.21	Styling and HTTP calls from the browser.
Docker Compose	n/a	Starts the NATS server with a single command.

CHAPTER SUMMARY

- Prompt Vault versions prompts: every change is a new version; one of them is active; nothing is lost.
- The business core has no screen and no HTTP: it talks through messages on a bus (NATS).
- The api is a thin translator between HTTP and NATS; the front end is a browser client of the api.
- The core has five areas (domain, port, app, adapter, infra). The next step is to understand why, after a little Python.

CHAPTER 2

2 Essential Python

The part of the language you need to read this project calmly

> **GUIDING QUESTION**
>
> Which Python features appear in the code, and what does each one mean?

You do not need to "know Python" to follow the project, only to recognize half a dozen ideas that repeat. We will meet them in the order they appear, always anchored in real code. (The api uses a few more, such as Pydantic models and generics; they are introduced in Chapter 7, where they are needed.)

2.1 The environment: uv, pyproject.toml and uv.lock

Python programs depend on third-party libraries. So that they do not get mixed with the libraries of other projects, every project has its own **virtual environment** (a `.venv` folder with an isolated "box" of libraries). `uv` is the tool that creates that box and takes care of it.

`core/pyproject.toml · the project's identity card`

```toml
[project]
name = "prompt-vault"
version = "0.1.0"
description = "Add your description here"
readme = "README.md"
requires-python = ">=3.13"
dependencies = [
    "aiosqlite>=0.22.1",
    "nats-py>=2.16.0",
    "rich>=15.0.0",
    "sqlalchemy[asyncio]>=2.1.8",
    "structlog>=26.1.0",
]
```

File / command	What it does
`pyproject.toml`	Declares the project name, the minimum Python version and the dependencies. You edit it (or let `uv` edit it).
`uv.lock`	Records the exact versions of everything that was installed, so anyone reproduces the same environment. Generated by `uv`; never edited by hand.
`uv sync`	Installs exactly what `uv.lock` says.
`uv add package`	Adds a dependency (updates `pyproject.toml` and `uv.lock`).
`uv run src/main.py`	Runs the program inside the project's environment.

The api has its own `pyproject.toml` and `uv.lock`, and its own `.venv`. Each Python project is independent: that is why the commands of Chapter 11 start with `cd core` or `cd api`.

LESSON LEARNED · A dependency that was missing

When SQLAlchemy 2.1 was turned on in asynchronous mode, the program failed: it requires the `greenlet` library, which the new version stopped installing by itself. The fix was to ask for the "extra" `sqlalchemy[asyncio]` (see the list above). Moral: an import error says exactly what is missing, so read the message to the end.

2.2 Modules, packages and imports

A `.py` file is a **module**. A folder with an `__init__.py` file is a **package**: a module that groups other modules. To use something from another place we use `import`:

Three ways to import (examples from the project)

```
from domain import Prompt                 # from the package 'domain', bring the class Prompt
from .status import PromptStatus          # one dot = 'the neighbor file of this one'
from ..events import PromptCreated        # two dots = 'the folder above'
```

IN THE PROJECT · Why does "uv run src/main.py" work?

When you run a script, Python puts the folder of that script in the list of places where it looks for modules. Since the script is `src/main.py`, the folder `src/` enters the search, and `import domain` finds `src/domain/`. To run anything else from the root of the project we use `PYTHONPATH=src`, which gives Python the same hint.

2.3 Types: saying what each thing is

In Python you can (and in this project we always do) annotate the expected type of each value. Python does not enforce it when running, but tools such as `pyright` check everything beforehand, and the code becomes its own documentation.

Written	Reading it in plain English
`name: str`	"name is a text"
`versions: list[PromptVersion]`	"versions is a list of PromptVersion"
`active_version_number: int | None`	"an integer or nothing (`None`)": here, "no active version"
`def delete(self) -> None`	"delete is a function that returns nothing"
`async def get(...) -> Prompt | None`	"an asynchronous function that returns a Prompt or `None`"
`dict[str, Any]`	"a dictionary whose keys are texts and whose values can be anything"

2.4 Classes, dataclasses and enums

A **class** is a mold for objects: it gathers data (attributes) and behaviors (methods). Inside it, `self` means "this very object". Four variations appear all the time:

- `@dataclass`: the `@` is a *decorator*, a sticker glued on the class. This one generates the constructor and the comparison automatically, saving repetitive code.
- `frozen=True`, inside `@dataclass`: makes the object immutable. Once created, it can never change.
- `Enum` / `StrEnum`: a closed list of valid options (such as the states of a prompt).
- `@property`: turns a method into something you read like an attribute and that cannot be changed from outside.

core/src/domain/entities/prompt_version.py

```python
@dataclass(frozen=True, slots=True)
class PromptVersion:
    """Immutable snapshot of a prompt's content.

    Versions are never edited or removed; a change in content is a new version.
    Which version is in use is decided by the owning Prompt, not by the version.
    """

    number: int
    content: str
    created_at: datetime
    id: UUID = field(default_factory=uuid4)

    def __post_init__(self) -> None:
        if self.number < 1:
            raise InvalidPromptError("Version number must be >= 1")
        if not self.content.strip():
            raise InvalidPromptError("Version content cannot be blank")
```

Read the listing as a sentence: "a `PromptVersion` is an immutable stamp with a number, a content, a date and an id; when it is created, it refuses a number lower than 1 and a blank content". That is exactly what the code says.

2.5 Protocol: contracts without inheritance

Python uses the idea of **duck typing**: "if it walks like a duck and quacks like a duck, it works as a duck". A `Protocol` describes the *shape* an object must have (which methods, with which signatures) without forcing anyone to inherit from anything. Any class that has those methods fulfils the contract.

core/src/port/prompt_repository.py · a contract

```python
class PromptRepository(Protocol):
    async def get(self, prompt_id: UUID) -> Prompt | None: ...

    async def save(self, prompt: Prompt) -> None:
        """Insert or update the whole aggregate, including all its versions."""
        ...

    async def list_active(self) -> Sequence[Prompt]:
        """Prompts whose status is ACTIVE."""
        ...

    async def list_deleted(self) -> Sequence[Prompt]:
        """Prompts whose status is DELETED (soft-deleted)."""
        ...
```

The `...` in place of a body means "here we only declare; the implementation comes from elsewhere". This technique is the basis of the *ports* we will see in Chapter 3.

2.6 async / await: the waiter who serves several tables

A good part of the time of a service is waiting: waiting for the database to answer, waiting for a message to arrive over the network. Instead of standing still, the program can serve another task while it waits. That is **asynchronous programming**.

> **ANALOGY · The waiter**
>
> A waiter who serves one table at a time, standing in front of the kitchen until the dish is ready, serves few people. An asynchronous waiter takes the order, goes to the next table and comes back when the kitchen calls. It is still one waiter, but nobody waits for nothing.

Word	Meaning
`async def f(): ...`	Defines a function "that can wait": a *coroutine*.
`await something()`	"I wait for the result, but meanwhile the program may serve other tasks."
`asyncio.run(main())`	Opens the "restaurant": starts the engine that coordinates all asynchronous tasks.
`asyncio.Lock()`	A "single key": only one task at a time can hold it (we will see why in Chapter 6).

> **IN THE PROJECT · Why is the project asynchronous?**
>
> Because its two sources of waiting, NATS (network) and SQLite (disk), have asynchronous libraries (`nats-py` and `aiosqlite`). The service then handles many messages without freezing. The api is asynchronous too: FastAPI runs your `async def` routes on the same kind of engine, and the browser front end has its own version of the idea (`async` / `await` and *promises* in TypeScript).

2.7 Context managers: "with" and "async with"

The `with` block guarantees that something is opened and closed properly, no matter what happens in the middle. Think of a door that closes by itself when you leave. In the project, the database transaction works like this:

Pattern repeated in the use cases (shortened)

```
async with self._uow:
    prompt = await require_prompt(self._uow.prompts, command.prompt_id)
    ...
    await self._uow.commit()
```

If anything goes wrong before `commit()`, when the block ends the transaction is undone automatically. We will come back to this slowly.

2.8 Exceptions: when something cannot continue

`raise` interrupts the function and signals a problem; `try / except` catches the signal to handle it. The project defines its own exceptions for each business rule that gets violated, for example `PromptDeletedError` ("you cannot do this to a deleted prompt"). Names like these make the error self-explanatory and allow handling it in a specific way.

2.9 A mini-dictionary of symbols

Symbol	Meaning
`_name`	Convention: "internal use, do not touch from outside" (e.g. `self._status`).
`__init__`	The constructor: runs when the object is created.
`*, activate: bool = False`	The `*` forces you to write `activate=True` by name; `= False` is the default value.
`{...}`	A dictionary (key: value pairs) or a set, depending on the content.
`f"text {x}"`	A text with embedded values (an *f-string*).
`x[T]` or `list[str]`	A *generic*: a container or wrapper that is parameterized by a type.
`[a for a in items if ...]`	A *comprehension*: builds a list by transforming and filtering another one.

CHECKPOINT

1. What is `uv.lock` for, and why do we not edit it by hand?
2. What does `int | None` mean in `active_version_number: int | None`?
3. What happens in `async with uow:` if an error occurs before `commit()`?

CHAPTER SUMMARY

- `uv` creates the environment; `pyproject.toml` declares dependencies; `uv.lock` pins versions.
- Folders with `__init__.py` are packages; `from x import y` brings things from other modules.
- Types (`str | None`) document the code and allow automatic checking.
- `dataclass`, `enum`, `property` and `Protocol` give shape to data and to contracts.
- `async/await` avoids idle waiting; `with` guarantees opening and closing; custom exceptions name the errors.

CHAPTER 3

3 Hexagonal architecture

Why the core has five folders and who may talk to whom

GUIDING QUESTION

Why is the core not "a bunch of files together", and which rule keeps the order?

3.1 The idea: protect the rules from the technology

Hexagonal architecture (also called *Ports and Adapters*, proposed by Alistair Cockburn) is born from an observation: technologies change, business rules last. Today we use NATS and SQLite; tomorrow it could be HTTP and PostgreSQL. If the rules ("only one active version", "deleting is soft") are mixed with the database code, every change of technology becomes a dangerous surgery.

ANALOGY · The wall socket and the travel adapter

Your laptop (the rules) does not know whether the energy comes from a hydroelectric plant or from a generator. It only knows the shape of the plug (the *port*). In every country you use a different adapter, and the laptop stays untouched. In software: the core defines the ports; the adapters plug the technologies in.

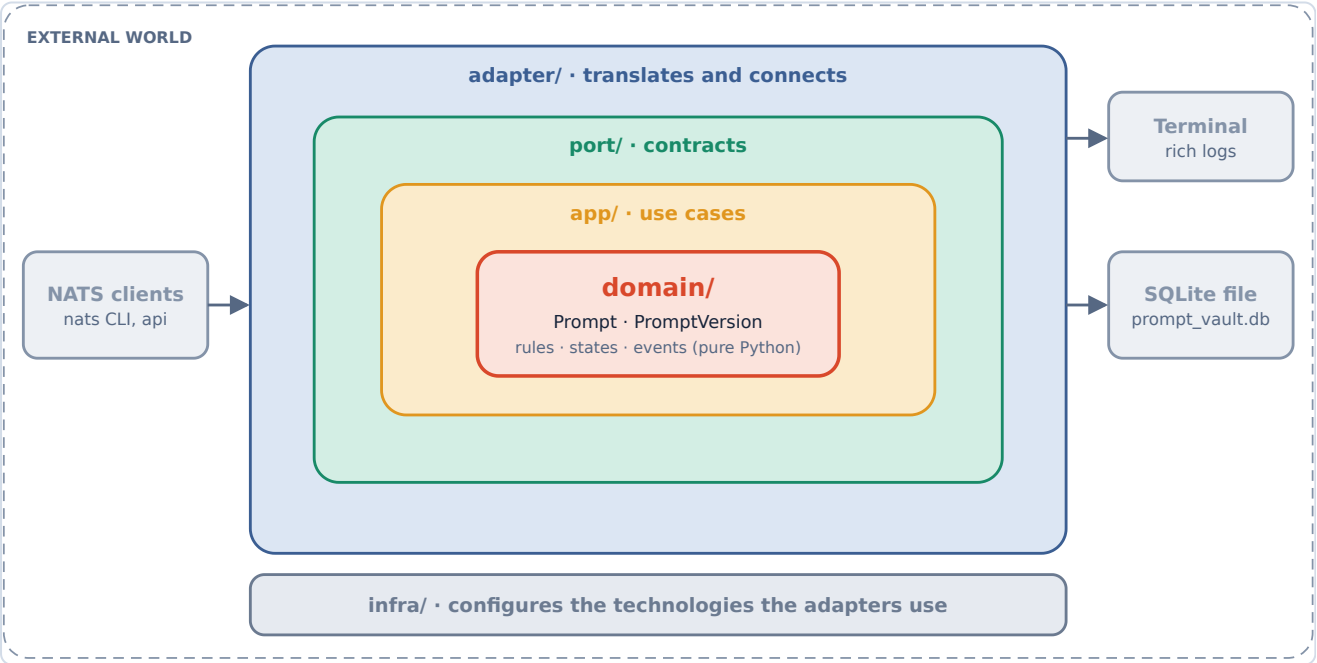


Figure 2 — The layers as rings: the core (domain) in the center, the external world at the edge. Everything that comes in or goes out crosses the rings.

Notice the whole system we saw in Chapter 1 sits at the border of this figure. The api, the `nats` CLI and the front end are all "NATS clients" in the left box; the SQLite file is the box on the right. The core never imports any of them.

3.2 The five blocks

An analogy helps to fix the role of each folder. Think of a restaurant:

Folder	Role	In the restaurant	Example in the project
`domain/`	The business rules, in pure Python	The recipes and the kitchen hygiene rules	`Prompt`, `PromptStatus`, `VersionDiff`
`app/`	Use cases: coordinates the flow	The chef who coordinates the preparation of each order	`CreateVersionUseCase`
`port/`	Contracts the core demands from the world	The order form and the supplier contract	`PromptRepository`, `UnitOfWork`
`adapter/`	Translates and connects technologies	The waiter (translates the customer) and the delivery person (supplier)	`SqlAlchemyPromptRepository`, `MessagingAdapter`
`infra/`	Only configures the technologies	The equipment: stove, fridge, phone	`create_engine`, `NatsClient`

There is a sixth character, outside the layer folders: `main.py`, the **composition root**. It is the only place that knows everyone and "connects the wires": it creates the adapters, hands them to the use cases and starts the service (Chapter 6).

3.3 The Dependency Rule

The whole discipline of the architecture fits in one sentence: **dependencies always point inward**. The adapters know the application and the domain; the application knows the domain and the ports; the domain knows nobody. "Knowing", here, literally means `import`.

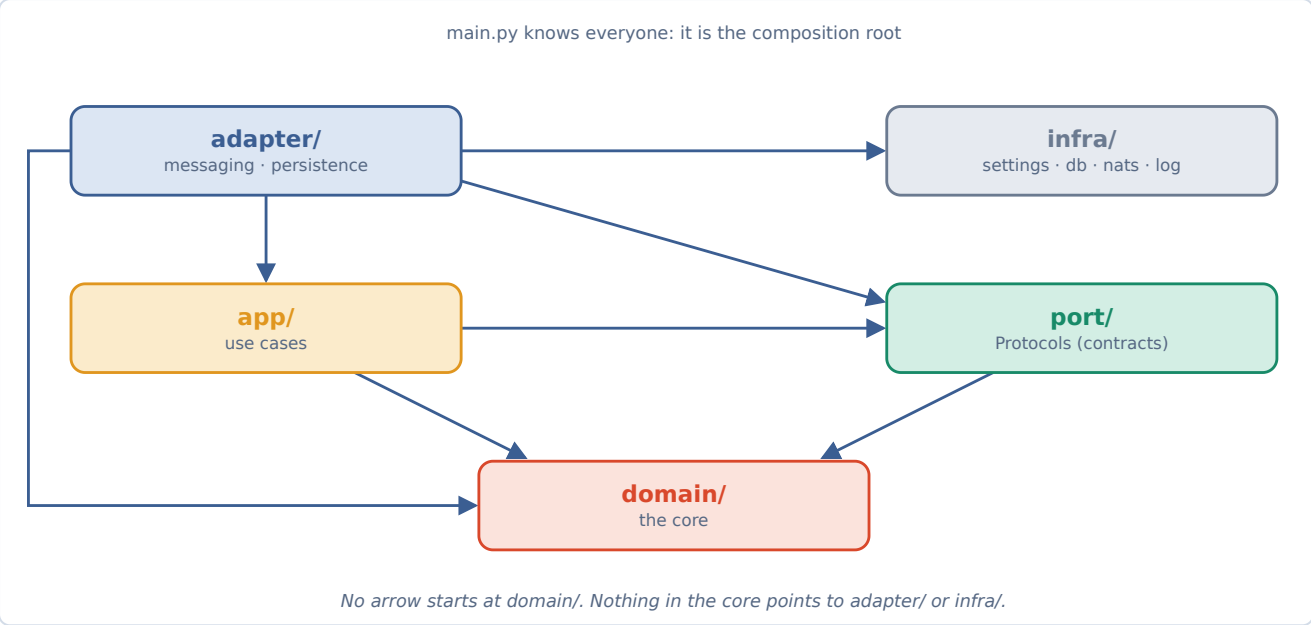


Figure 3 — The dependency rule. The arrows show who imports whom. Notice: no arrow starts in domain/.

IN THE PROJECT · This rule is verified, not just wished for

During development, we ran a search for any mention of `sqlalchemy`, `nats`, `structlog`, `rich`, `adapter`, `infra` and even `logging` inside `domain/`, `app/` and `port/`. The result was always "no occurrences". And the type checker (`pyright`) finished with 0 errors. The core really does not know that NATS or SQLite exist.

3.4 Why it is worth it (and what it costs)

Benefit	How it appeared in this project
Test without technology	The whole domain and every use case were tested with a fake in-memory repository: no database, no NATS.
Swap parts	Replacing SQLite with PostgreSQL would touch only `adapter/persistence` and the `infra/` configuration.
Evolve	Exposing the system over HTTP became a *new project* (`api/`) beside the messaging adapter, and the rules did not change.
Understand	Given a file, you know which layer it belongs to, and therefore what it may (and may not) do.

WATCH OUT · The cost

More folders and more "ceremony". For a 100-line script this would be overkill. Here it pays off because of the didactic nature of the project and because it was designed to grow: architecture is always a conscious trade.

■ 3.5 The patterns you will find

Architectures usually come with *design patterns*: named solutions for recurring problems. These are the ones in Prompt Vault:

Pattern	In one sentence	Where
Aggregate (DDD)	A group of objects treated as a unit, with a "root" that protects the rules.	`Prompt` (root) + `PromptVersion`
Repository	Hides the storage behind a "collection" interface.	`PromptRepository`
Unit of Work	Groups several operations into an "all or nothing" transaction.	`UnitOfWork`
Strategy	Swaps one algorithm for another without touching who uses it.	`DiffStrategy` / `LineDiffStrategy`
Factory	A function that manufactures ready-to-use objects.	use case factories in `main.py`
Dependency Injection	Whoever uses a piece receives it ready, instead of creating it.	constructors of the use cases
Command / Query	Separate what changes from what only reads.	`app/commands` and `app/queries`
Domain Events	Record "facts that happened" for anyone who wants to react.	`events.py` and `pull_events()`
Gateway / Facade	A thin entry point that hides a complex backend.	the `api/` project
DTO	A plain object that only carries data across a boundary.	`models/` in the api, `types.ts` in the front

CHECKPOINT

1. Why can `domain` not import `sqlalchemy`?
2. Which layer(s) would you change to swap SQLite for PostgreSQL?
3. What is the difference between a port and an adapter?

CHAPTER SUMMARY

- Hexagonal architecture separates rules (durable) from technology (replaceable).
- Five blocks: domain, app, port, adapter, infra, plus the `main.py` that wires them.
- Golden rule: dependencies point inward; the domain knows nobody.
- Port = contract (Protocol). Adapter = concrete implementation of that contract.

CHAPTER 4

4 Domain: the heart

Where the business rules live, and how they protect themselves

> **GUIDING QUESTION**
>
> Where do the rules of the system live, and what stops someone from breaking them?

The folder `core/src/domain/` has 605 lines and a rare property: nothing in it knows that a database, a network or a terminal exists. It is pure Python. That makes it easy to test and to understand; it is where the business "lives".

4.1 Rich modeling, not anemic

An **anemic model** is made only of data; the rules are scattered among whoever uses it. A **rich model** does the opposite: the object itself knows how to protect itself. Compare:

Illustration of the difference (not the full code)

```
prompt.status = "deleted"          # anemic: anyone changes it, whenever they want, checking nothing
prompt.delete()                    # rich: the Prompt validates the transition, stores the time,
records the event
```

All of Prompt Vault's domain follows the second approach: to change something, you *ask the object*, and it decides.

4.2 The aggregate: Prompt and its versions

In the jargon of Domain-Driven Design, an **aggregate** is a set of objects treated as a unit, with a *root* that is the only entry point. Here the root is `Prompt`: versions are only born by its hands. That is why the numbering of versions (1, 2, 3...) is never wrong: nobody else can create them.

Method of Prompt	What it does	Event recorded
`Prompt.create(name, content)`	Creates an active prompt, with version 1 active.	`PromptCreated`
`add_version(content, activate=False)`	Appends the next version; activates it only if asked.	`PromptVersionAdded` (+ `PromptVersionActivated`)
`activate_version(n)`	Starts serving version `n`. If it already is the active one, does nothing.	`PromptVersionActivated`
`deactivate()` / `reactivate()`	Switches the prompt off / on.	`PromptDeactivated` / `PromptReactivated`

Method of Prompt	What it does	Event recorded
`delete()` / `restore()`	Soft delete and recovery.	`PromptDeleted` / `PromptRestored`
`diff(a, b, strategy)`	Compares two versions. Only reads.	none
`Prompt.rehydrate(...)`	Rebuilds a prompt coming from storage, validating the rules.	none

core/src/domain/entities/prompt.py · add_version

```python
def add_version(self, content: str, *, activate: bool = False) -> PromptVersion:
    """Append a new version. It becomes the active one only if `activate` is set."""
    self._ensure_not_deleted("add a version to")
    now = _utcnow()
    version = PromptVersion(number=len(self._versions) + 1, content=content, created_at=now)
    self._versions.append(version)
    self._touch(now)
    self._record(
        PromptVersionAdded(prompt_id=self._id, occurred_at=now, version_number=version.number)
    )
    if activate:
        self.activate_version(version.number)
    return version
```

Look at the order: (1) refuse if the prompt is deleted; (2) create the version (which validates the content by itself); (3) store it; (4) update the date; (5) record the event; (6) only then activate, if requested. None of these decisions leaked out of the domain.

▪ 4.3 Immutable versions and the active-version pointer

There are two separate ideas, and mixing them up is the most common beginner's mistake: the **versions** (content, immutable) and the **pointer** that says which of them is "in force". Changing the active version only moves the pointer; nobody rewrites the past.

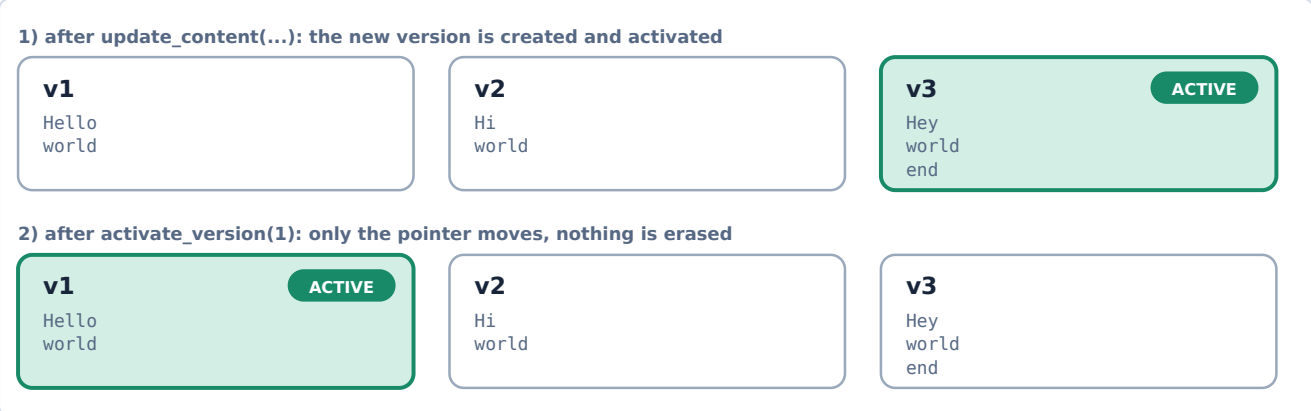


Figure 4 — Versions never change; only the 'active' pointer moves. Going back (rollback) is just pointing to an old version.

4.4 The life cycle: a state machine

A prompt is always in exactly one of three states, `ACTIVE`, `INACTIVE` or `DELETED`, and can only move from one to another along permitted paths. We call this a **state machine**. It is written explicitly, in a transition table anyone can read:

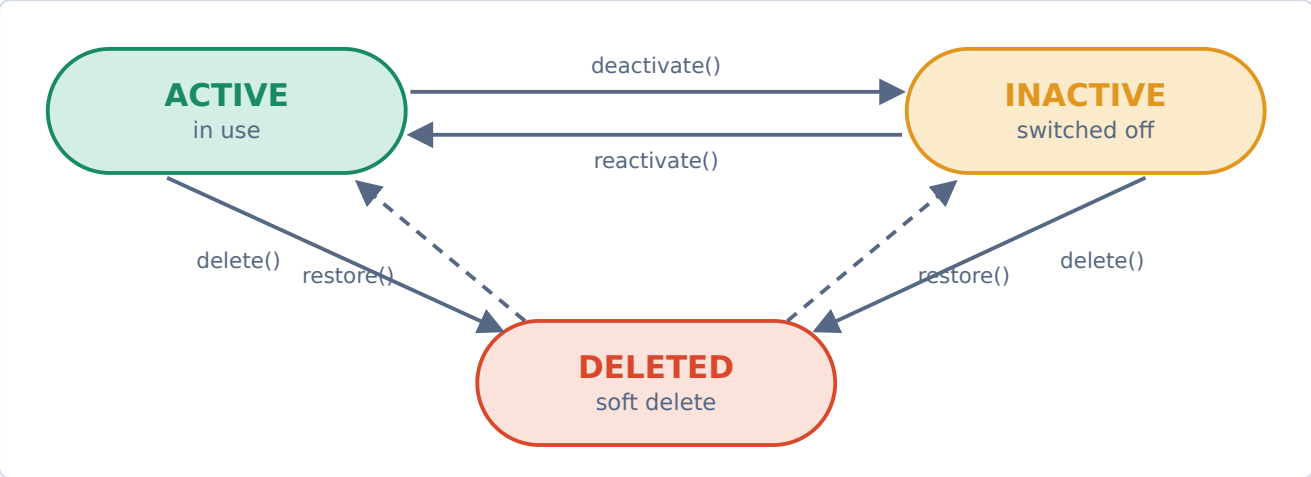


Figure 5 — The states and the permitted transitions. Only restore() leaves DELETED, and it returns to the state the prompt had before being deleted.

core/src/domain/status.py

```python
from enum import StrEnum


class PromptStatus(StrEnum):
    """Lifecycle state of a prompt."""

    ACTIVE = "active"
    INACTIVE = "inactive"
    DELETED = "deleted"

    def can_transition_to(self, target: "PromptStatus") -> bool:
        return target in _ALLOWED_TRANSITIONS[self]


# Explicit state machine. DELETED -> ACTIVE/INACTIVE is a restore (soft deletion).
_ALLOWED_TRANSITIONS: dict[PromptStatus, frozenset[PromptStatus]] = {
    PromptStatus.ACTIVE: frozenset({PromptStatus.INACTIVE, PromptStatus.DELETED}),
    PromptStatus.INACTIVE: frozenset({PromptStatus.ACTIVE, PromptStatus.DELETED}),
    PromptStatus.DELETED: frozenset({PromptStatus.ACTIVE, PromptStatus.INACTIVE}),
}
```

Deleting is *soft*: no row is destroyed. The prompt stores `deleted_at` and the state it was in before. When restored, it goes back exactly there:

core/src/domain/entities/prompt.py · delete and restore

```python
def delete(self) -> None:
    """Soft delete: all versions are kept and the prompt can be restored."""
    previous = self._status
    now = self._transition_to(PromptStatus.DELETED)
    self._status_before_deletion = previous
    self._deleted_at = now
    self._record(PromptDeleted(prompt_id=self._id, occurred_at=now, previous_status=previous))

def restore(self) -> None:
    """Undo a soft delete, going back to the status the prompt had before it."""
    if not self.is_deleted:
        raise PromptNotDeletedError()
    target = self._status_before_deletion or PromptStatus.INACTIVE
    now = self._transition_to(target)
    self._status_before_deletion = None
    self._deleted_at = None
    self._record(PromptRestored(prompt_id=self._id, occurred_at=now, restored_status=target))
```

4.5 Invariants: what is always true

An **invariant** is a statement that can never be false. The domain guarantees all of these; whoever tries to violate them receives a specific exception:

Invariant	Where it is guaranteed	Error
Name and content cannot be blank	`Prompt.__init__` and `PromptVersion.__post_init__`	`InvalidPromptError`
Versions numbered 1, 2, 3... with no gaps	`add_version` and `Prompt.__init__`	`InvalidPromptError`
The active version must exist	`activate_version` (via `get_version`)	`VersionNotFoundError`
A deleted prompt receives and activates no versions	`_ensure_not_deleted`	`PromptDeletedError`
Only what is deleted can be restored	`restore`	`PromptNotDeletedError`
Only permitted state transitions exist	`PromptStatus.can_transition_to`	`InvalidStateTransitionError`
`deleted_at` is filled if and only if the state is DELETED	`Prompt.__init__`	`InvalidPromptError`
Comparing requires two distinct, existing versions	`Prompt.diff`	`InvalidVersionComparisonError`

These exceptions matter beyond the core. In Chapter 6 you will see the messaging adapter give each of them a *code* in the reply (`version_not_found`, `prompt_not_deleted`, ...), and in Chapter 7 the api turn each code into an HTTP status.

4.6 Domain events: facts that happened

When something relevant happens, the aggregate records an **event**: a fact, in the past, immutable ("version 3 was activated"). Important: the domain only *records*; whoever decides what to do with the events (publish them on NATS, for example) is an outer layer, through the `pull_events()` method, which returns the list and empties it.

core/src/domain/events.py

```python
from dataclasses import dataclass
from datetime import datetime
from uuid import UUID

from .status import PromptStatus


@dataclass(frozen=True, slots=True, kw_only=True)
class DomainEvent:
    """Immutable fact about something that happened to a prompt."""

    prompt_id: UUID
    occurred_at: datetime


@dataclass(frozen=True, slots=True, kw_only=True)
class PromptCreated(DomainEvent):
    name: str
```

There are seven events: `PromptCreated`, `PromptVersionAdded`, `PromptVersionActivated`, `PromptDeactivated`, `PromptReactivated`, `PromptDeleted`, `PromptRestored`. They leave the system ready for an event-driven architecture: other services can react to changes without Prompt Vault knowing they exist.

4.7 The diff and the Strategy pattern

Comparing two versions is a business rule, so it lives in the domain. The result is a neutral object (no HTML, no colors, no screen), made of `DiffEntry` items (each line, with the kind of change: `ADDED`, `REMOVED`, `MODIFIED` or `UNCHANGED`). Whoever presents it decides how to show it. (The api and the front end, later, show it as lines of a split view.)

The comparison *algorithm*, however, is swappable. `Prompt.diff` receives a strategy: any object with a `name` and a `diff(old, new)` method (a Protocol, remember?):

core/src/domain/entities/prompt.py · diff

```python
def diff(self, from_number: int, to_number: int, strategy: DiffStrategy) -> VersionDiff:
    """Compare two distinct existing versions. Read-only: no state change, no events."""
    if from_number == to_number:
        raise InvalidVersionComparisonError(from_number)
    old = self.get_version(from_number)
    new = self.get_version(to_number)
    return VersionDiff(
        prompt_id=self._id,
        from_version=from_number,
        to_version=to_number,
        algorithm=strategy.name,
        entries=strategy.diff(old.content, new.content),
    )
```

The default strategy, `LineDiffStrategy`, compares line by line using the `difflib` module of the Python standard library. Its core is this loop, which translates the *operations* of `difflib` into entries of our model:

core/src/domain/diff/line_diff.py · the main loop

```python
for tag, i1, i2, j1, j2 in matcher.get_opcodes():
    if tag == "equal":
        for offset in range(i2 - i1):
            entries.append(self._unchanged(old_lines, new_lines, i1 + offset, j1 + offset))
        continue

    # "replace" pairs lines up as modifications; leftovers are plain removals/additions.
    paired = min(i2 - i1, j2 - j1) if tag == "replace" else 0
    for offset in range(paired):
        entries.append(
            DiffEntry(
                ChangeType.MODIFIED,
                old_line=i1 + offset + 1,
                new_line=j1 + offset + 1,
                old_text=old_lines[i1 + offset],
                new_text=new_lines[j1 + offset],
            )
        )
    for i in range(i1 + paired, i2):
        entries.append(
            DiffEntry(ChangeType.REMOVED, old_line=i + 1, old_text=old_lines[i])
        )
    for j in range(j1 + paired, j2):
        entries.append(DiffEntry(ChangeType.ADDED, new_line=j + 1, new_text=new_lines[j]))
```

A real example, executed on the system. Version 1: `line1 / line2 / line3 / line4`. Version 2: `line1 / line2 changed / line4 / line5`. The diff returned was:

Kind	Old line	New line	Old text	New text
unchanged	1	1	line1	line1
modified	2	2	line2	line2 changed
removed	3	none	line3	none
unchanged	4	3	line4	line4

Kind	Old line	New line	Old text	New text
added	none	4	none	line5

IN THE PROJECT · Extending without breaking anything

To create a "word by word" comparison, you only write a new class with `name` and `diff`, and hand it to the use case in `main.py`. The domain, the `CompareVersions` use case and the messaging adapter stay exactly as they are. That is the gain of the Strategy pattern.

4.8 A lesson the tests taught

LESSON LEARNED · The bug the smoke test found

In the first version of the domain, `restore()` worked even on prompts that were not deleted, and, since there was no stored previous state, it silently deactivated the prompt. Worse: `deactivate()` and `reactivate()` accepted deleted prompts, leaving a contradictory state (deleted but "active") that the domain itself refuses when loading, so the prompt would be saved and then become impossible to read.

The end-to-end test expected an error when restoring a non-deleted prompt and did not get one. The fix was three short guards (the `PromptNotDeletedError` and the `_ensure_not_deleted` you saw above). Moral: rules belong to the domain, and testing the system for real finds holes that reading the code does not show.

CHECKPOINT

1. Why is `PromptVersion` `frozen` (immutable)?
2. What happens if we call `restore()` on an active prompt?
3. What would you implement to compare two versions word by word?

CHAPTER SUMMARY

- The `domain` is pure Python: rules, states, events and the diff, with no technology.
- `Prompt` is the root of the aggregate; `PromptVersion` is immutable; only the active-version pointer moves.
- The life cycle is an explicit state machine; deleting is soft and reversible.
- Invariants are protected by specific exceptions; events record facts that happened.
- The diff uses the Strategy pattern: the algorithm is swappable without touching anything else.

CHAPTER 5

5 Ports and Application

Contracts and use cases: how a request becomes an action without depending on technology

GUIDING QUESTION

How does the core execute an action ("create a version") without knowing where the data lives or through which door the request arrived?

5.1 Ports: what the core demands from the world

The `port/` folder is tiny (57 lines) and very important. It contains three contracts, all written as Protocols. Each one expresses a need: "I need to store and find prompts", "I need to tell the world about events", "I need a transaction".

Port	The need	Methods
`PromptRepository`	Store and retrieve prompts as if they were a collection	`get`, `save`, `list_active`, `list_deleted`
`EventPublisher`	Tell the world about the domain events	`publish(events)`
`UnitOfWork`	Group operations into an "all or nothing" transaction	`prompts`, `commit`, `rollback`, `async with`

core/src/port/event_publisher.py

```python
from collections.abc import Sequence
from typing import Protocol

from domain import DomainEvent


class EventPublisher(Protocol):
    async def publish(self, events: Sequence[DomainEvent]) -> None: ...
```

core/src/port/unit_of_work.py

```python
"""Transactional boundary. Leaving the block without `commit()` must roll back."""

prompts: PromptRepository

async def __aenter__(self) -> Self: ...

async def __aexit__(
    self,
    exc_type: type[BaseException] | None,
    exc: BaseException | None,
    tb: TracebackType | None,
) -> None: ...

async def commit(self) -> None: ...

async def rollback(self) -> None: ...
```

CONCEPT · Minimal contracts

A port only has what the use cases really need. There is no `delete(id)` in the repository because deleting is a rule of the domain (`prompt.delete()`), followed by a save. The repository decides nothing: it only stores and fetches.

5.2 Unit of Work: the "all or nothing" transaction

ANALOGY · The bank transfer

Transferring 100 from account A to account B is two operations: debit A and credit B. If the power goes out between the two, the money would vanish. A *transaction* guarantees that either both happen or neither does. The Unit of Work is the object that opens this "protected box" and confirms (`commit`) or undoes (`rollback`) everything.

In the project: once you enter `async with uow:`, whatever is saved only counts if it reaches `commit()`. If an exception happens before it, leaving the block triggers the rollback and nothing stays stored.

5.3 Use cases: the orchestra

A **use case** is an action the system offers, from the point of view of whoever uses it: "create a prompt", "activate a version". The `app/` layer has 316 lines and contains exactly eleven use cases, split into two classic groups:

- **Commands (write):** change the state. They use the Unit of Work and publish events.
- **Queries (read):** only consult. They open no transaction and publish nothing.

Use case	Kind	Orchestrates	Returns
`CreatePrompt`	command	`Prompt.create`	the `Prompt`

Use case	Kind	Orchestrates	Returns
`CreateVersion`	command	`prompt.add_version(activate=...)`	the `Prompt` and the new `PromptVersion`
`UpdateContent`	command	`prompt.add_version(activate=True)`	the `Prompt` and the new `PromptVersion`
`ActivateVersion`	command	`prompt.activate_version(n)`	the `Prompt`
`DeletePrompt`	command	`prompt.delete()`	the `Prompt`
`RestorePrompt`	command	`prompt.restore()`	the `Prompt`
`GetPrompt`	query	repository `get`	the `Prompt`
`ListVersions`	query	`prompt.versions`	the `Prompt` (the adapter lists its versions)
`ListActivePrompts`	query	`list_active`	a list of prompts
`ListDeletedPrompts`	query	`list_deleted`	a list of prompts
`CompareVersions`	query	`prompt.diff(a, b, strategy)`	the `Prompt` and the `VersionDiff`

IN THE PROJECT · Use cases return the aggregate

In the first edition of this guide, `CreatePrompt` returned only an id and `ActivateVersion` returned nothing. Today every use case returns the **aggregate** (or the aggregate plus the object it created), and the adapter decides how to present it. Why? Because *presentation is not a business concern*. Returning the `Prompt` lets the adapter build the exact JSON the api and the front end need (ids, dates, which version is active) without the use case knowing anything about JSON.

5.4 Anatomy of a use case

core/src/app/commands/create_version.py

```python
class CreateVersionUseCase:
    def __init__(self, uow: UnitOfWork, publisher: EventPublisher) -> None:
        self._uow = uow
        self._publisher = publisher

    async def execute(self, command: CreateVersionCommand) -> tuple[Prompt, PromptVersion]:
        """Add a new version to an existing prompt. Returns the prompt and the new version."""
        async with self._uow:
            prompt = await require_prompt(self._uow.prompts, command.prompt_id)
            version = prompt.add_version(command.content, activate=command.activate)
            await self._uow.prompts.save(prompt)
            await self._uow.commit()
        await self._publisher.publish(prompt.pull_events())
        return prompt, version
```

1. The constructor receives the Unit of Work and the publisher. The use case creates nothing: it only uses what was handed to it.
2. `async with self._uow` opens the transaction.
3. `require_prompt` loads the prompt (or raises `PromptNotFoundError`).
4. `prompt.add_version(...)`: the business rule is called in the domain. The use case decides nothing.
5. `save` + `commit`: persists and confirms.
6. Only after the commit are the events published and the result returned.

> **WATCH OUT · Why publish after the commit?**
>
> If we published before, an error while saving would leave the world knowing about something that did not happen. The cost: if publishing fails *after* the commit, the data is already saved but the event is lost. A mature solution for that is the *outbox* pattern (recording the event in the same transaction), not implemented here and recorded as a next step.

5.5 Dependency injection and the power of the "fake"

Since the use case receives its dependencies (instead of creating them), we can hand it any object that fulfils the contract, including a fake made only for testing. That is how every use case was verified without a database and without NATS. This was the fake repository used in the tests:

Fake repository (verification script, outside the project)

```
class FakeRepo:
    def __init__(self): self.store = {}
    async def get(self, i): return self.store.get(i)
    async def save(self, p): self.store[p.id] = p; log.append("save")
    async def list_active(self): return [p for p in self.store.values() if p.status is
PromptStatus.ACTIVE]
    async def list_deleted(self): return [p for p in self.store.values() if p.is_deleted]
```

Six lines were enough to test the whole application. That is the prize of the architecture: the logic can be exercised in isolation. (The api uses the same trick: its tests replace the NATS publisher with a mock, as you will see in Chapter 10.)

> **IN THE PROJECT · Application errors**
>
> The domain does not know how to "fetch" prompts, it only receives them. That is why whoever raises `PromptNotFoundError` ("this id does not exist") is the `app` layer, in `require_prompt`. Business-rule errors (`DomainError` and children) cross the application untouched; the messaging adapter converts them into error codes.

CHECKPOINT

1. What is the difference between a command and a query?
2. Why are events only published after the commit?
3. Why does the use case receive the `uow` in the constructor instead of creating it?

CHAPTER SUMMARY

- Ports are three minimal contracts: repository, event publisher and unit of work.
- Use cases only coordinate: load, call the domain, save and publish. They return the aggregate.
- Commands write (with a transaction and events); queries only read.
- Dependency injection makes it possible to test with fake objects.

CHAPTER 6

6 Adapters and Infra

How the core connects to NATS, SQLite and the terminal

> **GUIDING QUESTION**
>
> How can a core that "knows no technology at all" store data and receive messages?

Here come the 1,027 lines that connect the core to the real world: 877 in the adapters and 150 in the infra. One golden rule applies to them: **no business rule**. Adapters only translate and connect; the infra only configures.

6.1 Infra: configuring the technologies

The `infra/` folder has four files, each configuring one technology. It imports nothing from the project: it is just tooling.

File	Responsibility
`settings.py`	Reads the configuration from environment variables, with default values.
`database.py`	Creates the SQLite engine, applies the concurrency settings and creates the tables.
`nats_client.py`	Opens and closes the connection to NATS and logs the connection events.
`logging.py`	Configures structlog and rich, once, in a single place.

Environment variable	Default	What it is for
`NATS_URL`	`nats://localhost:4222`	Address of the NATS server.
`DATABASE_URL`	`sqlite+aiosqlite:///./prompt_vault.db`	Where the database is (a single file).
`LOG_LEVEL`	`INFO`	How much detail appears in the logs.
`HANDLER_TIMEOUT_SECONDS`	`5`	Maximum time to process one message.

6.2 Persistence: SQLAlchemy and SQLite

SQLite and the WAL mode

SQLite is a database that fits in one file: there is no server to install. For concurrent use, we adjust a few options every time a connection is opened:

core/src/infra/database.py

```python
if engine.url.get_backend_name() == "sqlite":

    @event.listens_for(engine.sync_engine, "connect")
    def _configure_sqlite(dbapi_connection, _record) -> None:
        cursor = dbapi_connection.cursor()
        cursor.execute("PRAGMA journal_mode=WAL")  # readers don't block the writer
        cursor.execute("PRAGMA synchronous=NORMAL")  # safe with WAL, much faster
        cursor.execute("PRAGMA foreign_keys=ON")
        cursor.execute("PRAGMA busy_timeout=5000")  # wait for locks instead of failing
        cursor.close()
```

ANALOGY · The notebook with a scratch page (WAL)

In the ordinary mode, when someone writes in the "notebook", nobody can read it. In the WAL mode (*write-ahead log*), new notes go first to a scratch page beside it; readers keep reading the notebook while the writer takes notes. Later, the scratch page is merged. Result: reading and writing coexist. The other settings: `foreign_keys=ON` makes the database demand that the relations between tables are valid, and `busy_timeout` tells it to wait a little for a lock instead of failing immediately.

The ORM: tables as classes

An **ORM** (Object-Relational Mapper) such as SQLAlchemy lets you describe a table as a Python class. These classes are *not* the domain's: they are the relational model, and they never leave the adapter.

core/src/adapter/persistence/models.py

```python
class PromptModel(Base):
    __tablename__ = "prompts"

    id: Mapped[str] = mapped_column(String(36), primary_key=True)
    name: Mapped[str] = mapped_column(Text)
    status: Mapped[str] = mapped_column(String(16), index=True)
    active_version_number: Mapped[int | None] = mapped_column(Integer, nullable=True)
    status_before_deletion: Mapped[str | None] = mapped_column(String(16), nullable=True)
    created_at: Mapped[datetime] = mapped_column(UTCDateTime)
    updated_at: Mapped[datetime] = mapped_column(UTCDateTime)
    deleted_at: Mapped[datetime | None] = mapped_column(UTCDateTime, nullable=True)

    # lazy="raise": async sessions cannot lazy-load, so every query must eager-load explicitly.
    versions: Mapped[list["PromptVersionModel"]] = relationship(
        back_populates="prompt",
        order_by="PromptVersionModel.number",
        cascade="all, delete-orphan",
        lazy="raise",
    )
```

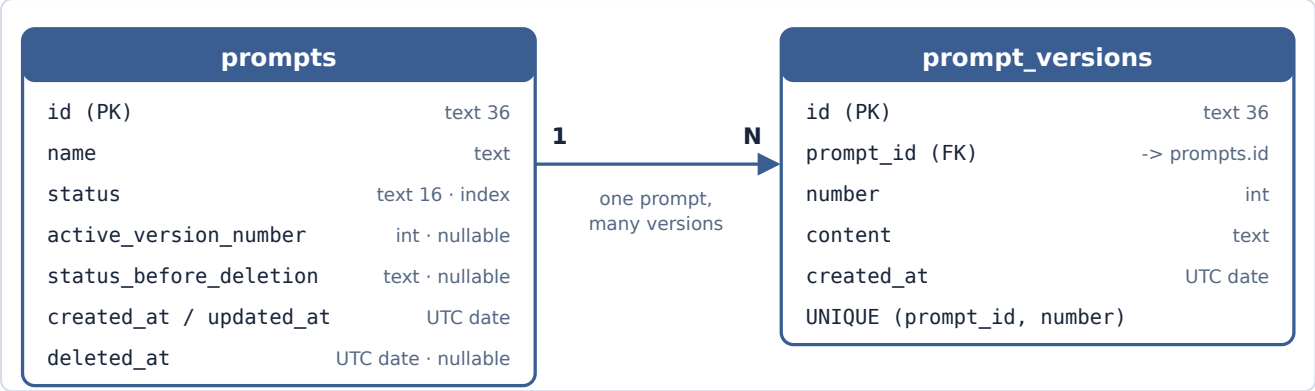


Figure 6 — The two tables. A prompt has many versions; the UNIQUE constraint prevents two equal numbers in the same prompt.

Domain is not a table: the mapper

Why two models? Because they serve different owners. The domain changes for business reasons; the tables, for storage reasons. The **mapper** does the translation in both directions. When reading, it rebuilds the aggregate with `Prompt.rehydrate`, which revalidates the rules and emits no events (reconstituting is not a new fact):

core/src/adapter/persistence/mapper.py

```python
def to_domain(row: PromptModel) -> Prompt:
    versions = [
        PromptVersion(
            id=UUID(version.id),
            number=version.number,
            content=version.content,
            created_at=version.created_at,
        )
        for version in row.versions
    ]
    status_before_deletion = row.status_before_deletion
    return Prompt.rehydrate(
        id=UUID(row.id),
        name=row.name,
        status=PromptStatus(row.status),
        versions=versions,
        active_version_number=row.active_version_number,
        created_at=row.created_at,
        updated_at=row.updated_at,
        deleted_at=row.deleted_at,
        status_before_deletion=(
            PromptStatus(status_before_deletion) if status_before_deletion else None
        ),
    )
```

When writing, the repository respects the nature of history: it only appends versions.

core/src/adapter/persistence/prompt_repository.py

```python
async def save(self, prompt: Prompt) -> None:
    row = await self._fetch_row(str(prompt.id))
    if row is None:
        self._session.add(to_new_row(prompt))
    else:
        apply_state(row, prompt)
        stored_numbers = {version.number for version in row.versions}
        # History is append-only: only versions that are not stored yet are inserted.
        for version in prompt.versions:
            if version.number not in stored_numbers:
                row.versions.append(version_to_row(prompt.id, version))
    await self._session.flush()
    log.info("prompt_saved", prompt_id=str(prompt.id), versions=len(prompt.versions))
```

IN THE PROJECT · A time zone detail

SQLite does not store time zones: it would hand back "loose" dates. A custom type, `UTCDateTime`, stores everything in UTC and returns time-zone-aware dates. A test confirmed it: the dates come back from the database identical to what the domain created.

The concrete Unit of Work and the write key

core/src/adapter/persistence/unit_of_work.py

```python
async def __aenter__(self) -> Self:
    await self._write_lock.acquire()
    try:
        self._session = self._session_factory()
        self.prompts = SqlAlchemyPromptRepository(self._session)
        self._committed = False
    except BaseException:
        self._write_lock.release()
        raise
    return self

async def __aexit__(
    self,
    exc_type: type[BaseException] | None,
    exc: BaseException | None,
    tb: TracebackType | None,
) -> None:
    session = self._require_session()
    try:
        if not self._committed:
            await session.rollback()
            log.info("transaction_rolled_back", failed=exc_type is not None)
    finally:
        await session.close()
        self._session = None
        self._write_lock.release()

async def commit(self) -> None:
```

ANALOGY · Two people, one spreadsheet

If two people open the same spreadsheet, each changes a cell and saves, whoever saves last wipes out the other's change: the so-called *lost update*. The `asyncio.Lock` in `__aenter__` is the "single key to the room": while a write transaction is open, the others wait in line. We tested it with 40 simultaneous commands on the same prompt: the versions came out numbered 2 to 41, with no loss and no duplicate.

WATCH OUT · An honest concession

Queries do not need a transaction, so they use a repository that opens a short session on every call (`SessionPerCallPromptRepository`). Since the port requires `save`, it exists but raises `NotImplementedError`. The "clean" solution would be a read-only port (`PromptReader`), touching `port/` and the queries. It was left as a future improvement.

6.3 Messaging: the NATS adapter

NATS is a messaging system: a central server (the *broker*) where clients publish messages on **subjects** and others subscribe to those subjects. Names use dots to form hierarchies (`prompt.events.created`) and the wildcard `>` means "everything below" (`prompt.events.>` catches all the events). There are three usage patterns, all present here:



Figure 7 — The three conversation patterns in Prompt Vault.

The subjects of the system

Kind	Subjects	Mode
Commands	`prompt.create`, `prompt.version.create`, `prompt.content.update`, `prompt.version.activate`, `prompt.delete`, `prompt.restore`	Asynchronous. They reply if the sender provides a reply address.
Queries	`prompt.get`, `prompt.versions.list`, `prompt.list.active`, `prompt.list.deleted`, `prompt.versions.compare`	Request and reply. Without a reply address, they are ignored.

Kind	Subjects	Mode
Events (outbound)	`prompt.events.created`, `...version_added`, `...version_activated`, `...deleted`, `...restored`, `...deactivated`, `...reactivated`	Publish and subscribe.
Errors (outbound)	`prompt.errors`	Errors of commands that had nobody to reply to.

The correlation id: the tracking code

> **ANALOGY · The postal tracking code**
>
> With a tracking code, you follow the parcel through every stage. The correlation id does the same with a message: it enters through the `X-Correlation-Id` header (or the system creates one) and accompanies the reply, the published events and every log line. Later, you only search for that code to rebuild the whole story.

core/src/adapter/messaging/correlation.py

```python
def correlation_scope(correlation_id: str) -> Iterator[None]:
    """Make the id available to the publisher and to every log line inside the block."""
    token = _current.set(correlation_id)
    structlog.contextvars.bind_contextvars(correlation_id=correlation_id)
    try:
        yield
    finally:
        structlog.contextvars.unbind_contextvars("correlation_id")
        _current.reset(token)
```

It uses a `ContextVar`: a variable whose value is "private" to each asynchronous task. That way, simultaneous messages do not mix up their ids.

The adapter only translates

See the "handler" of a command. It validates the format (is it a UUID? is it text?), builds the command and calls the use case. Then it asks a *serializer* to turn the result into the JSON the outside world expects. No business rule:

core/src/adapter/messaging/handlers.py · create_version

```python
async def create_version(p: dict[str, Any]) -> Any:
    command = CreateVersionCommand(
        prompt_id=require_uuid(p, "prompt_id"),
        content=require_str(p, "content"),
        activate=optional_bool(p, "activate"),
    )
    prompt, version = await factories.create_version().execute(command)
    return version_response(prompt, version)
```

core/src/adapter/messaging/serializers.py · the shape of a prompt

```python
def prompt_response(prompt: Prompt, *, content: str | None = None) -> dict[str, Any]:
    """PromptResponse. `content` is only sent by create_prompt."""
    active = prompt.active_version
    data: dict[str, Any] = {
        "id": str(prompt.id),
        "name": prompt.name,
        "status": prompt.status.value,
        "active_version_id": str(active.id) if active else None,
        "created_at": prompt.created_at.isoformat(),
        "updated_at": prompt.updated_at.isoformat(),
    }
    if content is not None:
        data["content"] = content
    return data
```

The serializers are the *contract* of the core with the outside world. Their output matches the DTOs documented by the api (`PromptResponse`, `PromptVersionResponse`, `PromptDetailResponse`, `VersionDiffResponse`), which is why the api can forward the `data` untouched and the front end can trust it. A "modified" line of the domain diff, for instance, becomes two lines in the contract: a `removed` and an `added`, like a unified diff.

Successes, failures and delays

The heart of the adapter decides what to do with successes, failures and slowness. Read carefully the `asyncio.wait_for` (the timeout) and the separation between "expected" errors (a warning) and "unexpected" ones (an exception, with traceback):

core/src/adapter/messaging/server.py

```python
async def _execute(
    self,
    route: Route,
    payload: dict[str, Any],
    decode_error: InvalidPayloadError | None,
    correlation_id: str,
) -> tuple[bool, bytes]:
    try:
        if decode_error is not None:
            raise decode_error
        log.info("use_case_started", use_case=route.use_case)
        data = await asyncio.wait_for(route.handler(payload), timeout=self._timeout)
        log.info("use_case_completed", use_case=route.use_case)
        return True, ok_envelope(correlation_id, data)
    except Exception as exc:
        code, message, unexpected = to_error(exc)
        if unexpected:
            log.exception("use_case_failed", use_case=route.use_case, error_code=code)
        else:
            log.warning("use_case_rejected", use_case=route.use_case, error_code=code,
error=message)
        return False, error_envelope(correlation_id, code, message, subject=route.subject)
```

The envelope and the error codes

Every reply is an envelope `{ok, correlation_id, data | error}`. A real error, produced by forcing a timeout:

Real error reply

```
{"ok": false, "correlation_id": "ae85212c-327d-4d49-a689-64493a259c1f",
 "error": {"code": "timeout", "message": "The operation timed out"}, "subject":
"prompt.list.active"}
```

core/src/adapter/messaging/errors.py

```python
def to_error(exc: BaseException) -> tuple[str, str, bool]:
    """Map an exception to (code, client-safe message, is_unexpected)."""
    if isinstance(exc, InvalidPayloadError):
        return "invalid_payload", str(exc), False
    if isinstance(exc, PromptNotFoundError):
        return "not_found", str(exc), False
    if isinstance(exc, DomainError):
        return _domain_code(exc), str(exc), False
    if isinstance(exc, TimeoutError):
        return "timeout", "The operation timed out", True
    return "internal_error", "Internal error", True  # details stay in the logs


_DOMAIN_CODES: tuple[tuple[type[DomainError], str], ...] = (
    (VersionNotFoundError, "version_not_found"),
    (InvalidVersionComparisonError, "invalid_version_comparison"),
    (PromptDeletedError, "prompt_deleted"),
    (PromptNotDeletedError, "prompt_not_deleted"),
    (InvalidPromptError, "invalid_prompt"),
)


def _domain_code(exc: DomainError) -> str:
    """One code per business rule; anything else keeps the generic `domain_error`."""
    for error_type, code in _DOMAIN_CODES:
        if isinstance(exc, error_type):
            return code
    return "domain_error"
```

Code	When it happens	Example
`invalid_payload`	Malformed message: invalid JSON, missing or wrongly typed field, malformed UUID.	`{not json`
`not_found`	The requested prompt does not exist.	a random UUID
`version_not_found`	The version number does not exist in that prompt.	activating version 99
`invalid_version_comparison`	Comparing a version with itself.	`from_version == to_version`
`prompt_deleted`	The operation is not allowed on a deleted prompt.	activating a version of a deleted prompt

Code	When it happens	Example
`prompt_not_deleted`	Restoring a prompt that is not deleted.	`restore` on an active prompt
`invalid_prompt`	Invalid data for a prompt or version.	a blank name
`domain_error`	Any other business rule.	the generic fallback
`timeout`	Processing exceeded the limit.	a slow handler
`internal_error`	Unexpected error. Details only in the log, never in the reply.	an unforeseen failure

> **LESSON LEARNED · One code per rule**
>
> In the first edition every business rule became the same `domain_error`, so a client had to read the English message to know what happened. When the api was built, that turned out to be fragile: the api guessed the exception by matching text, and a reworded message would silently degrade the answer. The core now sends **one code per rule**, and the api maps codes, not words (the text match survives only as a fallback for `domain_error`). Moral: an interface between two systems should be a contract of codes, never of prose.

Publishing the events

core/src/adapter/messaging/event_publisher.py

```python
async def publish(self, events: Sequence[DomainEvent]) -> None:
    correlation_id = get_correlation_id()
    headers = {CORRELATION_HEADER: correlation_id} if correlation_id else None
    for event in events:
        name = _EVENT_NAMES.get(type(event), type(event).__name__.lower())
        subject = f"{subjects.EVENTS_PREFIX}.{name}"
        body: dict[str, Any] = {"event": name, **dataclasses.asdict(event)}
        try:
            await self._nats_client.nc.publish(subject, encode(body), headers=headers)
            log.info("event_published", subject=subject, prompt_id=str(event.prompt_id))
        except Exception:
            log.exception("event_publish_failed", subject=subject)
```

Notice the `try / except`: if publishing fails, the error is logged but not propagated. The reason: the data was already saved; telling the client that the command failed would make it try again and duplicate the effect.

▪ 6.4 Structured logging: structlog + rich

A traditional log is a loose sentence: "user 42 deleted the prompt". To find something you have to guess the text. A **structured log** separates the event from its fields (`event=use_case_started use_case=DeletePrompt correlation_id=fail-1`), and each field becomes searchable. `structlog`

produces that; `rich` takes care of the presentation in the terminal. The `correlation_id` is added automatically to every line, thanks to the `ContextVar` we saw.

This is the real trace of a command that failed (deleting a non-existent prompt, with no reply address), condensed to fit the page:

Real log (condensed)

```
INFO       event_received            subject=prompt.delete kind=command has_reply=False
correlation_id=fail-1
INFO       use_case_started          use_case=DeletePrompt
correlation_id=fail-1
INFO       transaction_rolled_back   failed=True
correlation_id=fail-1
WARNING    use_case_rejected         use_case=DeletePrompt error_code=not_found correlation_id=fail-
1
INFO       error_published           subject=prompt.errors source=prompt.delete correlation_id=fail-
1
```

Read from top to bottom: the message arrived, the use case started, the transaction was undone (nothing stored), the error was classified as `not_found` and published on `prompt.errors`, all with the same `correlation_id`. The domain does not import the logger: logs live only in the adapters.

LESSON LEARNED · When rich gets in the way

While testing the timeout, the error line came out as a raw dictionary instead of the normal format. The cause: the `RichHandler`, with tracebacks on, ignores structlog's formatter when there is an exception. The fix was to turn that option off and let structlog itself format the traceback (also with rich). Moral: integrating two good libraries requires testing the junction, including the error path.

6.5 main.py: the composition root

All of this is loose pieces until someone wires them. That someone is `main.py`, the composition root: the only file that knows all the layers.

#	Step	Why
1	Configure logging and read the settings	First of all, to record whatever comes next.
2	Create the SQLite engine and the tables	The database must exist before the first message.
3	Connect to NATS	If it fails, the program logs the error and exits.
4	Create the publisher, the write lock and the read repository	Pieces shared by all messages.
5	Build the use case factories	One new instance per message (see below).
6	Start the `MessagingAdapter` (subscribe to the subjects)	From here on the service answers.
7	Wait for SIGINT/SIGTERM and shut down cleanly	Cancel subscriptions, drain NATS, close the database.

core/src/main.py · the factories

```
def uow() -> SqlAlchemyUnitOfWork:
    return SqlAlchemyUnitOfWork(session_factory, write_lock)

factories = UseCaseFactories(
    create_prompt=lambda: CreatePromptUseCase(uow(), publisher),
    create_version=lambda: CreateVersionUseCase(uow(), publisher),
    update_content=lambda: UpdateContentUseCase(uow(), publisher),
    activate_version=lambda: ActivateVersionUseCase(uow(), publisher),
    delete_prompt=lambda: DeletePromptUseCase(uow(), publisher),
    restore_prompt=lambda: RestorePromptUseCase(uow(), publisher),
    get_prompt=lambda: GetPromptUseCase(reader),
    list_versions=lambda: ListVersionsUseCase(reader),
    list_active_prompts=lambda: ListActivePromptsUseCase(reader),
    list_deleted_prompts=lambda: ListDeletedPromptsUseCase(reader),
    compare_versions=lambda: CompareVersionsUseCase(reader, diff_strategy),
)
adapter = MessagingAdapter(
    nats_client, build_routes(factories), settings.handler_timeout_seconds
)
```

CONCEPT · Why "one instance per message"?

Every write use case carries its Unit of Work, and therefore its database session. A session cannot be shared by simultaneous messages. That is why the `lambda` (an anonymous function) manufactures a new use case, with its own session, for each message received.

CHECKPOINT

1. What is the correlation id for?
2. Why does the app create a new instance of each use case for each message?
3. What does the SQLite WAL mode improve?

CHAPTER SUMMARY

- `infra` only configures; adapters only translate and connect: no business rule.
- Persistence: a relational model separate from the domain, with a mapper; history only grows; a lock prevents lost updates.
- Messaging: subjects for commands and queries, a standard envelope, one error code per rule, a timeout and a correlation id.
- Serializers shape the replies into the contract the api and the front end rely on.
- Structured logs with structlog + rich; the correlation id stitches everything together.
- `main.py` wires the pieces: it is the only one that knows all the layers.

CHAPTER 7

7 The HTTP API

How a web request reaches the core, and how errors find their way back

GUIDING QUESTION

If the core only understands NATS messages, how does a browser, which only speaks HTTP, use it?

The `api/` project is a **gateway**: a thin entry point that hides a backend. It has no business rules. Its whole job is to (1) accept an HTTP request, (2) check that its shape is valid, (3) send one NATS message to the core, and (4) turn the reply, or the failure, into an HTTP response. It has about 660 lines of application code and 695 lines of tests.

ANALOGY · The receptionist

In a clinic, the receptionist does not treat patients. They greet you, check that your form is filled in correctly, pass it to the doctor's room and bring the answer back. If the doctor is busy or absent, the receptionist explains it politely instead of letting you wander into the corridor. That is the api: it knows the *format* of every conversation but none of the medicine.

7.1 The layers of the api

The api uses the familiar MVC-style folders, but notice that each folder has a very small job:

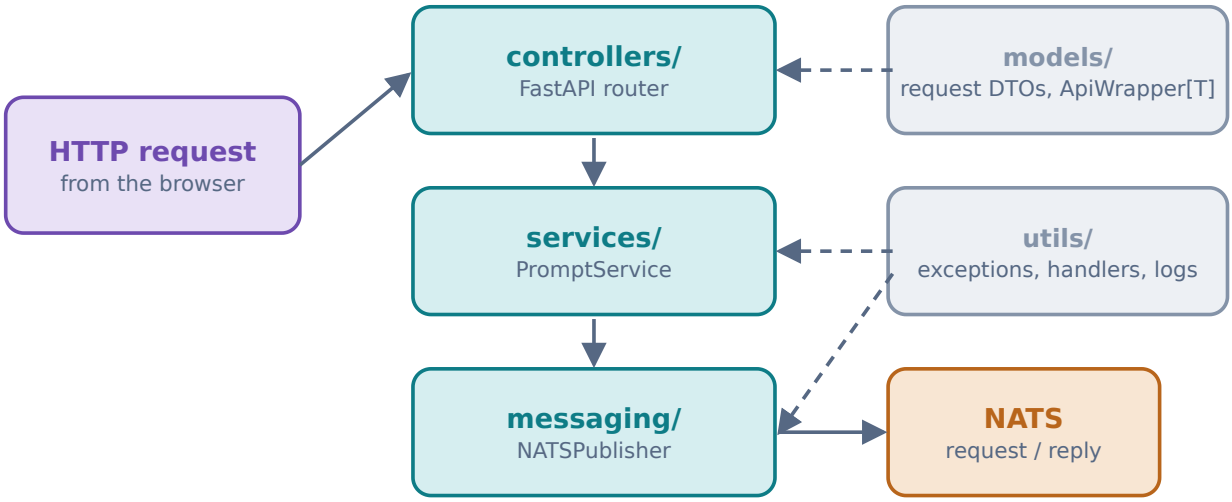


Figure 8 — The layers of the api. Only messaging/ imports nats-py; controllers never catch errors.

Folder	Job	Example
`controllers/`	Declares the HTTP routes and turns the request into a DTO. No logic.	`update_content(prompt_id, body, service)`
`services/`	One method per use case: builds the payload and picks the NATS subject.	`PromptService.update_content`
`messaging/`	The *only* code that talks to NATS. Sends the request, unwraps the reply, raises errors.	`NATSPublisher.request`
`models/`	DTOs (Pydantic) for requests, and the models that *document* the replies.	`CreatePromptRequest`, `ApiWrapper[T]`
`utils/`	Exceptions, the error handlers, JSON helpers and logging.	`BaseAppException`
`tests/`	117 unit tests with NATS mocked.	`test_nats_publisher.py`

The entry point wires it together. Read `main.py` slowly; it is the whole application:

api/main.py

```python
import os
from collections.abc import AsyncIterator
from contextlib import asynccontextmanager

import nats
from fastapi import FastAPI

from controllers.prompt_controller import router as prompt_router
from messaging.nats_publisher import NATSPublisher
from services.prompt_service import PromptService
from utils.error_handlers import register_exception_handlers
from utils.logger import setup_logging

NATS_URL = os.getenv("NATS_URL", "nats://localhost:4222")

setup_logging()


@asynccontextmanager
async def lifespan(app: FastAPI) -> AsyncIterator[None]:
    nc = await nats.connect(NATS_URL)
    app.state.prompt_service = PromptService(NATSPublisher(nc))
    try:
        yield
    finally:
        await nc.drain()


app = FastAPI(title="Prompt Vault API", lifespan=lifespan)
app.include_router(prompt_router)

register_exception_handlers(app)
```

Three ideas appear here. **The lifespan** is code that runs when the server starts (connect to NATS, build the service) and when it stops (`drain()`: finish what is in flight, then close). **`app.state`** is a small shelf where the app keeps shared objects. **`register_exception_handlers`** installs the central error handling of section 7.5.

■ 7.2 FastAPI in five minutes

FastAPI is a web framework. You declare a route with a decorator, and the function's parameters tell it what the request must contain:

api/controllers/prompt_controller.py · a route

```python
async def list_versions(prompt_id: UUID, service: PromptService = Service) -> Any:
    return await service.list_versions(PromptIdRequest(prompt_id=prompt_id))


@router.put("/{prompt_id}/content", responses={200: ok(PromptVersionResponse)})
async def update_content(
    prompt_id: UUID, body: UpdateContentBody, service: PromptService = Service
) -> Any:
    return await service.update_content(
        UpdateContentRequest(prompt_id=prompt_id, content=body.content)
```

- `@router.put("/{prompt_id}/content")` binds the HTTP method and path.
- `prompt_id: UUID` is read from the path **and validated**: an id that is not a UUID is rejected with `422` before the function even runs.
- `body: UpdateContentBody` is a Pydantic model: FastAPI reads the JSON body and checks it against the model.
- `service: PromptService = Service` is **dependency injection**: FastAPI calls `get_prompt_service` and hands the result in. In the tests we replace that provider with a mock, without touching the route.
- The function body is one line: call the service. Nothing else.

> **CONCEPT · Pydantic models are DTOs**
>
> A **DTO** (data transfer object) is a plain object that only carries data across a boundary. Pydantic turns a class with type annotations into a validator: `CreatePromptRequest(name: str, content: str)` refuses a body without `content`. That validation is the *shape* check. Whether the name is blank is a *business* rule, so it stays in the core.

7.3 The service builds the message

api/services/prompt_service.py

```python
class PromptService:
    """Use cases: build the payload, pick the subject, hand it to the publisher."""

    def __init__(self, publisher: NATSPublisher) -> None:
        self.publisher = publisher

    # ---- commands

    async def create_prompt(self, req: CreatePromptRequest) -> Any:
        return await self.publisher.request(
            PROMPT_CREATE, {"name": req.name, "content": req.content}
        )

    async def add_version(self, req: AddVersionRequest) -> Any:
        return await self.publisher.request(
            PROMPT_VERSION_CREATE,
            {"prompt_id": str(req.prompt_id), "content": req.content},
        )

    async def update_content(self, req: UpdateContentRequest) -> Any:
        return await self.publisher.request(
            PROMPT_CONTENT_UPDATE,
            {"prompt_id": str(req.prompt_id), "content": req.content},
        )

    async def activate_version(self, req: ActivateVersionRequest) -> Any:
        return await self.publisher.request(
            PROMPT_VERSION_ACTIVATE,
            {"prompt_id": str(req.prompt_id), "version_number": req.version_number},
        )

    async def soft_delete_prompt(self, req: PromptIdRequest) -> Any:
```

Each method does the same three things: build a payload (with ids turned into strings so they are valid JSON), choose the subject, and ask the publisher. The subject names are the very same ones the core listens to; the api and the core agree on them as a contract. Before this phase the commands used a fire-and-forget publish. They now use *request and reply*, because a browser needs an answer (the id of the created prompt, the new version number) and needs to hear about errors.

7.4 The publisher: the only door to NATS

api/messaging/nats_publisher.py

```python
class NATSPublisher:
    """The only place in the API that talks to NATS. Raises only BaseAppException."""

    def __init__(self, nc: NATSClient) -> None:
        self.nc = nc

    async def publish(self, subject: str, payload: dict[str, Any]) -> None:
        """Fire-and-forget."""
        try:
            await self.nc.publish(subject, encode_json(payload))
        except (NATSError, OSError, TypeError, ValueError) as exc:
            raise NATSPublishException(str(exc)) from exc

    async def request(
        self, subject: str, payload: dict[str, Any], timeout: float = 3.0
    ) -> Any:
        """Request/reply. Returns the `data` of the Core's wrapper or raises a BaseAppException."""
        try:
            msg = await self.nc.request(subject, encode_json(payload), timeout=timeout)
        except (NATSTimeoutError, TimeoutError) as exc:
            raise CoreTimeoutException() from exc
        except NoRespondersError as exc:
            raise CoreUnavailableException() from exc
        except (NATSError, OSError, TypeError, ValueError) as exc:
            raise NATSPublishException(str(exc)) from exc

        envelope = decode_json_object(msg.data)
        try:
            wrapper = ApiWrapper[Any].model_validate(envelope)
        except ValidationError as exc:
            raise CoreResponseException("invalid envelope") from exc
        if not wrapper.ok:
            error = envelope.get("error")
            error = error if isinstance(error, dict) else {}
            raise _core_error(
                str(error.get("code", "internal_error")),
                str(error.get("message", "")),
                payload,
            )
        return wrapper.data
```

Read `request` as a story:

1. **Send** the request and wait up to `timeout` seconds. If NATS is slow, the api raises `CoreTimeoutException`. If nobody is subscribed to that subject (the core is down), NATS answers "no responders" and the api raises `CoreUnavailableException`.
2. **Decode** the reply bytes as JSON. Garbage becomes `CoreResponseException`.
3. **Validate the envelope** with `ApiWrapper[Any]` (below). A reply without `ok` is a protocol violation.
4. **If `ok` is false**, turn the core's error code into an exception of the api.
5. **Otherwise return only `data`**. The controller returns it as the HTTP body, untouched.

That last point is a design principle worth stating: **the gateway transports, it does not reshape**. It never rebuilds a prompt, never renames fields, never "fixes" dates. The core produces the JSON contract; the api forwards it. This keeps domain logic from being duplicated in two places.

api/models/response_models.py · the generic wrapper

```python
class ApiWrapper[T](BaseModel):
    """Envelope every Core reply comes in. Use ApiWrapper[PromptResponse], ApiWrapper[list[...]] etc."""

    ok: bool
    data: T | None = None
```

`ApiWrapper[T]` is a **generic** model: `T` is a placeholder for "whatever the data is". `ApiWrapper[PromptResponse]` wraps a prompt, `ApiWrapper[list[PromptVersionResponse]]` wraps a list. The same class describes every reply of the core. (The other models in that file, such as `PromptResponse` and `VersionDiffResponse`, document the shape of `data`. FastAPI shows them in Swagger. They do not validate the actual response, because the gateway does not reshape it; the global smoke test of Chapter 10 checks that the core really produces them.)

7.5 Errors: one base class, one central place

Every error the api raises on purpose is a subclass of one base class:

api/utils/exceptions.py

```python
class BaseAppException(Exception):
    """Base for every error the API raises on purpose."""

    def __init__(
        self,
        message: str,
        error_code: str,
        http_status: int = 400,
        details: dict[str, Any] | None = None,
    ) -> None:
        super().__init__(message)
        self.message = message
        self.error_code = error_code
        self.http_status = http_status
        self.details = details or {}


# ---- domain

class PromptNotFoundException(BaseAppException):
    def __init__(self, prompt_id: str | None = None) -> None:
        label = f"Prompt {prompt_id}" if prompt_id else "Prompt"
        super().__init__(f"{label} não encontrado", "PROMPT_NOT_FOUND", 404)
```

An exception carries four things: a human `message`, a stable `error_code`, the `http_status` to answer with, and optional `details`. There is one subclass per situation:

HTTP	Error codes (all subclasses of `BaseAppException`)
404	`PROMPT_NOT_FOUND`, `VERSION_NOT_FOUND`
409	`PROMPT_ALREADY_DELETED`, `PROMPT_NOT_DELETED`, `DOMAIN_RULE_VIOLATION`
422	`INVALID_REQUEST` (bad body, bad UUID, or bad payload at the core), `INVALID_VERSION_COMPARISON`
502	`CORE_RESPONSE_ERROR` (the core answered nonsense)
503	`NATS_PUBLISH_FAILED`, `CORE_UNAVAILABLE`
504	`CORE_TIMEOUT`
500	`INTERNAL_SERVER_ERROR` (anything unforeseen)

The mapping from a *core error code* to one of these exceptions lives in one function, `_core_error`, using a `match` statement (Python's pattern matching):

api/messaging/nats_publisher.py · from core code to exception

```python
def _core_error(code: str, message: str, payload: dict[str, Any]) -> BaseAppException:
    match code:
        case "not_found":
            return PromptNotFoundException(payload.get("prompt_id"))
        case "invalid_payload":
            return InvalidRequestException({"reason": message})
        case "version_not_found":
            return VersionNotFoundException({"reason": message})
        case "invalid_version_comparison":
            return InvalidVersionComparisonException({"reason": message})
        case "prompt_deleted":
            return PromptAlreadyDeletedException({"reason": message})
        case "prompt_not_deleted":
            return PromptNotDeletedException({"reason": message})
        case "invalid_prompt":
            return DomainRuleException({"reason": message})
        case "domain_error":
            return _domain_error(message)
        case "timeout":
            return CoreTimeoutException()
        case _:
            return CoreResponseException(f"internal core error ({code})")
```

And a single place turns every exception into the JSON the client sees:

api/utils/error_handlers.py

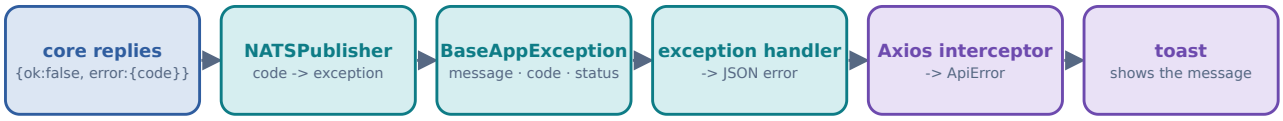
```python
def _error(status: int, code: str, message: str, details: dict | None = None) -> JSONResponse:
    return JSONResponse(
        status_code=status,
        content={"error": {"code": code, "message": message, "details": details or {}}},
    )


async def app_exception_handler(_: Request, exc: BaseAppException) -> JSONResponse:
    log.warning("app_error code=%s message=%s", exc.error_code, exc.message)
    return _error(exc.http_status, exc.error_code, exc.message, exc.details)


async def validation_exception_handler(_: Request, exc: RequestValidationError) -> JSONResponse:
    fields = [
        {"field": ".".join(str(p) for p in err["loc"]), "error": err["msg"]}
        for err in exc.errors()
    ]
    invalid = InvalidRequestException({"fields": fields})
    return _error(invalid.http_status, invalid.error_code, invalid.message, invalid.details)


async def generic_exception_handler(_: Request, exc: Exception) -> JSONResponse:
    log.error("unhandled_error", exc_info=exc)  # the traceback stays in the logs
    return _error(500, "INTERNAL_SERVER_ERROR", "Erro interno do servidor")


def register_exception_handlers(app: FastAPI) -> None:
    app.add_exception_handler(BaseAppException, app_exception_handler)
    app.add_exception_handler(RequestValidationError, validation_exception_handler)
    app.add_exception_handler(Exception, generic_exception_handler)
```



Controllers and services never catch exceptions: the error travels untouched to the handler.

Figure 9 — The path of an error, from the core's reply to a message on the screen.

IN THE PROJECT · Controllers do not catch exceptions

Look back at the route in section 7.2: there is no `try`. When something goes wrong deep inside, the exception simply travels upward until FastAPI hands it to the registered handler. That is intentional. Error handling is a *cross-cutting concern*, and putting it in one place guarantees every route answers errors in the same format: `{"error": {"code", "message", "details"}}`. A stack trace is **never** sent to the client; the generic handler logs it and answers with a harmless "Internal server error".

WATCH OUT · Language of the messages

The `message` field of the api errors (and the texts of the front end) are written in Portuguese, because they are what the end user reads. Everything else, from code and comments to the error *codes* themselves, is in English. A client should rely on the `code`, never on the message, which is meant for people.

▪ 7.6 The contract: Swagger and OpenAPI

FastAPI generates an **OpenAPI** document from the routes and models: a machine-readable description of every endpoint, request body and response. The interactive page at `/docs` (Swagger UI) is drawn from it. The front end keeps a snapshot of that document and generates its TypeScript types from it (Chapter 8), so a change in the api contract becomes a compile error in the front end instead of a surprise in production.

▪ 7.7 Testing the api without a network

The api has 117 unit tests, grouped by layer: `models`, `messaging`, `services`, `controllers` and `utils`. They never open a network connection:

Layer	What the tests replace	What they check
`NATSPublisher`	The NATS client, with an `AsyncMock`	Subject and JSON bytes sent; every failure becomes the right exception; every core error code maps to the right exception.
`PromptService`	The publisher, with a mock	Each of the 11 methods sends exactly one message, with the exact subject and payload.
Controllers	`PromptService`, via FastAPI's dependency override	Each route returns 200 with the service's data; bad input returns the standard 422; service errors become the standard JSON.
`utils`	Nothing: they are pure	Exceptions carry the right code and status; handlers never leak internals.

CHECKPOINT

1. Why does `NATSPublisher.request` return only `wrapper.data` instead of the whole reply?
2. Why do the controllers have no `try / except`?
3. Which three things can make the api answer `503` or `504` without any bug in the code?

CHAPTER SUMMARY

- The api is a gateway: it validates shape, sends one NATS request per use case and returns the core's `data` untouched.
- Only `messaging/` talks to NATS; controllers stay one line long.
- Errors are subclasses of `BaseAppException`, created from the core's error codes, and rendered by central handlers into one JSON format.
- FastAPI publishes the contract as OpenAPI; the front end builds its types from it.
- Everything is unit-tested with NATS mocked.

CHAPTER 8

8 The front end

How the browser application turns the api into screens

> **GUIDING QUESTION**
>
> How does a web page ask the api for data, keep it in sync with what the user does, and show errors without breaking?

The `front/` project is a **single-page application** (SPA): the browser downloads one HTML page and one JavaScript bundle, and from then on JavaScript draws every screen and talks to the api in the background. It is about 1,900 lines of TypeScript. This chapter assumes you have never used React; the next section gives the five ideas you need.

8.1 React in five minutes

Idea	What it means	Example in the project
Component	A function that returns a piece of screen (JSX, which looks like HTML inside TypeScript).	`PromptCard` draws one card.
Props	The inputs of a component, like function arguments.	`<PromptCard prompt={p} onDetails={...} />`
State	Data a component remembers between draws. Changing it redraws the component.	`const [query, setQuery] = useState("")`
Effect	Code that runs after a draw, usually to fetch data or subscribe to something. Its cleanup function runs before the next effect.	`useEffect(() => { loadVersions() }, [promptId])`
Lifting state up	When two components need the same data, the parent owns it and passes it down.	`App` owns the list of prompts.

TypeScript adds types to JavaScript, the same idea you met in Chapter 2: `prompt: PromptItem` documents what a component receives, and the compiler refuses a wrong call. The project is built with **Vite** (a fast development server and bundler), styled with **Tailwind CSS** (utility classes written right in the markup) and calls the api with **Axios** (an HTTP client library).

8.2 The map of the front end

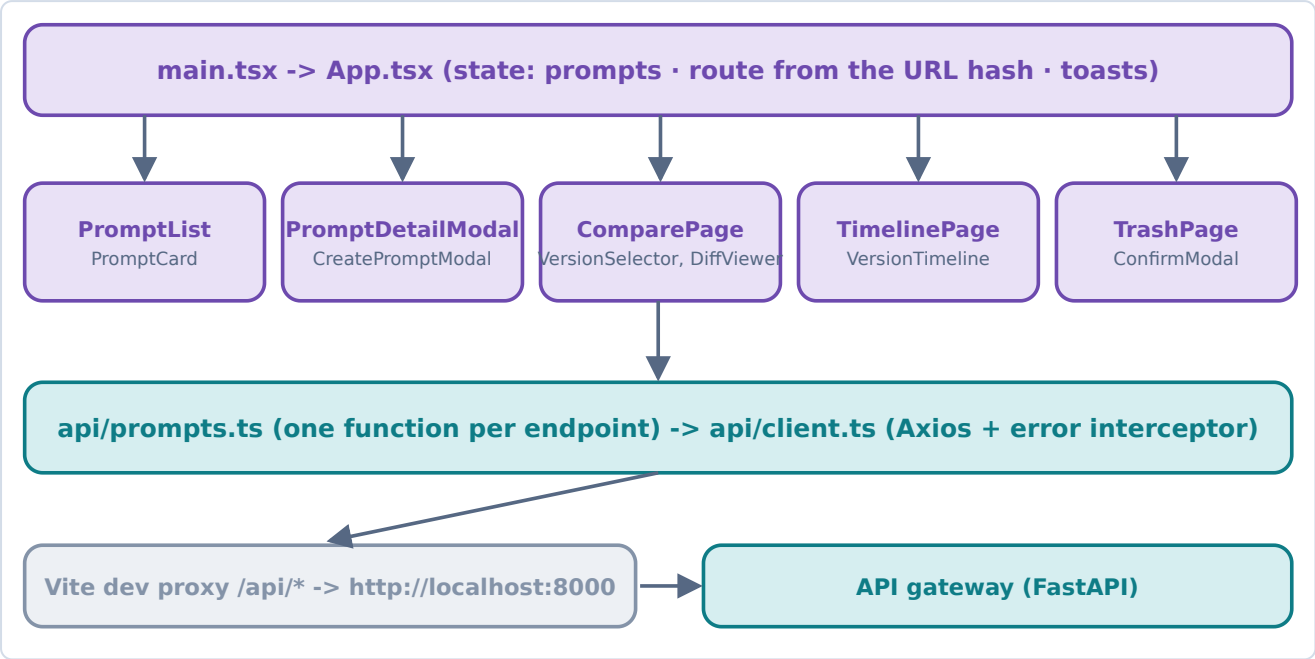


Figure 10 — The front end: App owns the state and the route; screens are components; one thin layer talks HTTP.

Read it top-down. `main.tsx` starts React and renders `App`. `App` keeps three things: the list of prompts, the current **route** (derived from the URL), and the **toasts** (the small notifications). It chooses which screen to show. Every screen reaches the api only through `api/prompts.ts`, and that file goes through one shared Axios instance in `api/client.ts`.

8.3 The service layer: Axios and the contract

Every call to the api lives in one file, one function per endpoint:

front/src/api/prompts.ts · excerpt

```
/** GET /prompts/{prompt_id} */
export const getPrompt = async (promptId: string): Promise<PromptDetailResponse> =>
  (await http.get<PromptDetailResponse>(`/prompts/${promptId}`)).data;

/** POST /prompts/{prompt_id}/activate: makes that version the one in use (rollback or promote). */
export const activateVersion = async (promptId: string, versionNumber: number):
Promise<PromptResponse> =>
  (await http.post<PromptResponse>(`/prompts/${promptId}/activate`, { version_number: versionNumber
})).data;
```

Where do the types `PromptDetailResponse` and `PromptResponse` come from? **From the api itself.** The api publishes an OpenAPI document (Chapter 7); the front end keeps a snapshot of it (`openapi.json`) and runs the tool `openapi-typescript` to generate `src/api/schema.d.ts`. If the api changes a field name, regenerating the types makes the front end *fail to compile* exactly where it used the old name. The contract is enforced by the compiler.

The shared Axios instance also normalizes errors, so that every screen can treat a failure the same way:

front/src/api/client.ts

```ts
import axios, { AxiosError } from "axios";

/** Error shape returned by the gateway: { error: { code, message, details } }. */
interface GatewayError {
  error?: { code?: string; message?: string; details?: Record<string, unknown> };
}

export class ApiError extends Error {
  constructor(
    message: string,
    readonly code: string,
    readonly status?: number,
  ) {
    super(message);
    this.name = "ApiError";
  }
}

export const http = axios.create({
  baseURL: import.meta.env.VITE_API_BASE_URL ?? "/api",
  timeout: 15_000,
});

http.interceptors.response.use(
  (response) => response,
  (error: AxiosError<GatewayError>) => {
    const apiError = error.response?.data?.error;
    if (apiError?.message) {
      return Promise.reject(new ApiError(apiError.message, apiError.code ?? "UNKNOWN",
error.response?.status));
    }
    if (!error.response) {
      return Promise.reject(new ApiError("Não foi possível conectar à API.", "NETWORK_ERROR"));
    }
    return Promise.reject(
      new ApiError(`Erro inesperado (HTTP ${error.response.status}).`, "UNEXPECTED",
error.response.status),
    );
  },
);

export const errorMessage = (error: unknown): string =>
  error instanceof Error ? error.message : "Ocorreu um erro inesperado.";
```

The interceptor reads the standard error JSON of the api (`{error: {code, message}}`) and throws an `ApiError` carrying the message. If there is no response at all (the api is down), it throws a "could not connect" error. Screens never parse HTTP errors themselves.

CONCEPT · CORS and the Vite proxy

Browsers refuse, by default, to let a page served from `localhost:5173` call an API on `localhost:8000`: that is the *same-origin policy*, relaxed only if the API opts in with CORS headers. The api has no CORS configuration. In development, Vite solves this with a **proxy**: the front end calls `/api/...` on its own address, and Vite forwards the request to the api, removing the `/api` prefix. From the browser's point of view there is only one server.

8.4 State and routing in App

There is no router library. The current screen is encoded in the URL **hash** (the part after `#`), and a few lines parse it:

front/src/App.tsx · a tiny hash router

```
/**
 * Tiny hash router: "#/compare", "#/compare/{id}", "#/compare/{id}?from=1&to=3" and
 * "#/timeline/{id}" and "#/trash". Everything else is the list.
 */
function parseRoute(): Route {
  const compare = location.hash.match(new RegExp(`^#/compare(?:/${ID})?/?(?:\\?from=(\\d+)&to=
(\\d+))?$`));
  if (compare) {
    return {
      name: "compare",
      promptId: compare[1],
      from: compare[2] ? Number(compare[2]) : undefined,
      to: compare[3] ? Number(compare[3]) : undefined,
    };
  }
  const timeline = location.hash.match(new RegExp(`^#/timeline/${ID}/?$`));
  if (timeline) return { name: "timeline", promptId: timeline[1] };
  return /^#\/trash\/?$/.test(location.hash) ? { name: "trash" } : { name: "list" };
}

function useRoute(): Route {
  const [route, setRoute] = useState(parseRoute);
  useEffect(() => {
    const onChange = () => setRoute(parseRoute());
    window.addEventListener("hashchange", onChange);
    return () => window.removeEventListener("hashchange", onChange);
  }, []);
  return route;
}
```

`useRoute` listens for the browser's `hashchange` event and redraws when the URL changes. Because the screen is in the URL, the back button works, and a link such as `#/timeline/<id>` can be bookmarked or shared. These are the screens:

Route	Screen
`#/`	The list of prompts: search, create, details.

Route	Screen
`#/compare`, `#/compare/{id}`, `#/compare/{id}?from=1&to=3`	Compare two versions. The prompt and versions can be pre-selected from a link.
`#/timeline/{id}`	The version history of one prompt.
`#/trash`	Deleted prompts, with recovery.

8.5 Loading data: why one request per card

The list endpoint, `GET /prompts/active`, returns a *summary* of each prompt: name, status, dates and the id of the active version. It does **not** include the content or the version number. But a card must show "v3 active" and a preview of the active content. So after the list arrives, the front end asks for the detail of every prompt in parallel:

front/src/App.tsx · loadPrompts (shortened)

```
const summaries = await listActivePrompts();
const details = await Promise.allSettled(summaries.map((s) => getPrompt(s.id)));
// fulfilled -> use the detail; rejected -> keep the summary and warn the user
```

`Promise.allSettled` is the important choice: if one detail fails, the others still arrive, and the card of the failed one simply shows "version unavailable". It is a trade-off. For a handful of prompts it is fine; for hundreds it would be many requests, and the right fix would be on the api side (a list endpoint that includes the active version). Honest limitations like this are worth writing down.

After a change, the front end does not refetch everything; it reloads just the prompt that changed:

front/src/App.tsx · reloadOne

```
const reloadOne = useCallback(async (promptId: string) => {
  const detail = await getPrompt(promptId);
  setPrompts((all) => [detail, ...all.filter((prompt) => prompt.id !== promptId)]);
}, []);
```

`reloadOne` fetches one prompt and puts it at the top of the list, replacing the old copy. It is passed down to the screens as a callback, which is how a timeline page can tell the main list "this prompt changed".

8.6 The screens

The list and the cards

`PromptList` owns the search box (a state variable) and filters the prompts by name on the client, in memory. Each `PromptCard` shows the name, a badge for the active version, a preview of the active content (`line-clamp` cuts it to three lines) and three icons: timeline, compare and details.

Create, edit, activate

Creating a prompt opens a modal with a name and a content field, sends `POST /prompts` and inserts the result into the list. The details modal shows the active content, with **Copy** and **Edit** buttons. Editing offers two ways to save, matching two different endpoints:

front/src/components/PromptDetailModal.tsx · two ways to save

```
/** "Salvar e ativar" creates the version and makes it active; "sem ativar" only creates it. */
const save = async (activate: boolean) => {
  if (!changed || saving) return;
  setSaving(activate ? "activate" : "draft");
  try {
    const version = activate ? await updateContent(prompt.id, draft) : await addVersion(prompt.id,
draft);
    await onSaved(prompt.id);
    onNotify(
      "success",
      activate
        ? `Nova versão v${version.version_number} salva e ativada.`
        : `Versão v${version.version_number} criada (não ativada).`,
    );
```

"Save and activate" calls `PUT .../content` (the new version becomes the active one). "Save without activating" calls `POST .../versions` (the version is kept in the history and the current one stays in force). The difference is the one Chapter 4 called "versions" versus "the pointer".

Comparing versions

The compare screen has three parts: a prompt selector, a pair of version selectors, and the diff viewer. Whenever the prompt or either version changes, it asks the api for the diff. Two subtleties deserve attention.

First, the **stale response guard**. If the user changes the selection quickly, two requests are in flight, and the *older* one might answer *last*. Without protection, the screen would show a diff that no longer matches the selectors. Each effect therefore marks itself as current, and its cleanup function marks it as stale:

front/src/components/ComparePage.tsx · ignoring stale answers

```
// 3. The diff itself, always from the API. A newer selection discards an older answer.
useEffect(() => {
  if (!promptId || !detail || before === null || after === null || before === after) {
    return;
  }
  let current = true;
  setDiff({ status: "loading" });
  compareVersions(promptId, before, after)
    .then((data) => current && setDiff({ status: "ready", data }))
    .catch((error) => {
      if (!current) return;
```

Second, the **split view**. The api returns a flat list of lines, each `added`, `removed` or `unchanged`. A side-by-side view needs rows with a left and a right cell. A tiny algorithm pairs each run of removed lines with the added lines that follow it:

front/src/components/DiffViewer.tsx · pairing lines for the split view

```tsx
/** Pairs each run of removed lines with the added lines that follow it, side by side. */
function toRows(lines: DiffLineResponse[]): Row[] {
  const rows: Row[] = [];
  let i = 0;
  while (i < lines.length) {
    const line = lines[i];
    if (line.kind === "unchanged") {
      rows.push({ left: line, right: line });
      i += 1;
      continue;
    }
    const removed: DiffLineResponse[] = [];
    const added: DiffLineResponse[] = [];
    while (i < lines.length && lines[i].kind !== "unchanged") {
      (lines[i].kind === "removed" ? removed : added).push(lines[i]);
      i += 1;
    }
    for (let k = 0; k < Math.max(removed.length, added.length); k += 1) {
      rows.push({ left: removed[k], right: added[k] });
    }
  }
  return rows;
}
```

Notice what the front end does *not* do: it does not compute the diff. The comparison algorithm lives in the core (Chapter 4); the front end only arranges the result on screen. The unified view simply lists the lines in order. Additions are green, removals red, and long lines scroll horizontally instead of wrapping or overlapping.

The timeline

The timeline lists the versions newest first, like a commit history. Each node shows the version number, the date, a two-line summary and a highlighted badge for the active one. The summary is a small pure function:

front/src/components/VersionTimeline.tsx · summarize

```tsx
/** The first non-empty lines (shortened) plus how many more lines the version has. */
function summarize(content: string): { lines: string[]; more: number } {
  const filled = content.split("\n").map((line) => line.trim()).filter((line) => line !== "");
  const shown = filled.slice(0, SUMMARY_LINES).map((line) => (line.length > SUMMARY_MAX ?
`${line.slice(0, SUMMARY_MAX - 1).trimEnd()}…` : line));
  return { lines: shown, more: filled.length - shown.length };
}
```

Clicking a node expands its full content. Each non-active node also offers **Activate this version**, which is how the user performs a rollback:

front/src/components/TimelinePage.tsx · activating a version

```
/** Activates a version, then reloads the timeline in place (no skeleton) and the main list. */
const activate = async (version: PromptVersionResponse) => {
  try {
    await activateVersion(promptId, version.version_number);
    setState({ status: "ready", versions: newestFirst(await listVersions(promptId)) });
    await onChanged(promptId).catch(() => undefined);
    onNotify("success", `v${version.version_number} agora é a versão ativa.`);
  } catch (error) {
    onNotify("error", errorMessage(error));
  }
};
```

After activating, the page reloads its versions *in place* (no loading skeleton, so the highlight simply moves), then tells the main list to refresh that prompt, then shows a toast. The two quick links on each node, *compare with previous* and *compare with active*, are plain links to `#/compare/{id}?from=..&to=..`.

The trash

Deleting a prompt (with a confirmation dialog) calls `DELETE /prompts/{id}`, a soft delete in the core. The trash screen lists `GET /prompts/deleted` and offers **Recover**, which calls `POST /prompts/{id}/recover`. The date shown is the prompt's `updated_at`, which the core sets at the moment of deletion; the api exposes no separate "deleted at" field, so the front end does not invent one.

■ 8.7 Patterns that make it feel solid

- **Every asynchronous screen has four states:** loading (a skeleton that mimics the final layout), empty (a friendly message), error (the message from the api, plus a "Try again" button) and ready.
- **Toasts** give feedback for actions: green for success, red for errors (they stay longer).
- **Modals** trap the Escape key and block closing while a request is in flight, so a double click cannot send two requests. A confirmation modal opened *on top of* another modal closes alone on Escape.
- **Dark and light themes** are a CSS class on the page root. The choice is saved in `localStorage`, the only thing the front end persists. A tiny script in `index.html` applies it before the first paint, avoiding a flash of the wrong theme.
- **Accessibility:** buttons that are only icons have text labels for screen readers (`aria-label`), dialogs declare `role="dialog"` and `aria-modal`, and loading regions announce themselves with `aria-busy`.

LESSON LEARNED · The test that failed for the wrong reason

While testing the timeline in a real browser, four checks failed. The page was right; the test was wrong: it counted the loading skeleton's placeholders as if they were the real nodes, and it searched for the word "active", which also appears in the "Compare with active" links. Browser tests are code, and flaky selectors are bugs. The fix was to wait for something only the real content has (a `<time>` element) and to match the exact badge text. A failing test deserves the same suspicion as a failing feature.

WATCH OUT · What the front end does not have yet

There are no automated front-end tests: it is checked by the TypeScript compiler (`npm run build`) and by exercising the real browser against the real api, which was done repeatedly while building it. Pagination of long lists, an "undo" after deleting and deleting directly from a card are also left for later.

CHECKPOINT

1. Why does the front end call `/api/...` instead of `http://localhost:8000/...` directly?
2. What problem does the "stale response guard" in `ComparePage` solve?
3. Why does the front end not compute the diff itself?

CHAPTER SUMMARY

- The front end is a single-page app: React components, TypeScript types, Tailwind styling and Axios for HTTP.
- One service layer wraps the endpoints; its types are generated from the api's OpenAPI document, so the compiler enforces the contract.
- The screen is encoded in the URL hash; `App` owns the list, the route and the toasts.
- The core produces the diff; the front end only arranges it (split or unified).
- Every async screen handles loading, empty, error and ready; stale answers are ignored.

CHAPTER 9

9 A request end to end

What happens, step by step, when someone clicks Save in the browser

> **GUIDING QUESTION**
>
> When I click a button, through which pieces does the request pass, and in what order?

Now that you know every part, let us stitch everything together following one single action: the user edits a prompt in the browser and clicks **Save and activate**. Along the way we will cross three projects, three borders (HTTP, NATS, SQL) and every layer of the core.

9.1 The core's half of the story

First the inside view, the part executed by the core. A client (here the api, but it could be the `nats` CLI) sends a message to `prompt.content.update`:

The request, as the NATS CLI would send it

```
nats request prompt.content.update \
  '{"prompt_id": "<ID>", "content": "Hey\nworld\nend"}' \
  -H "X-Correlation-Id:my-trace"
```

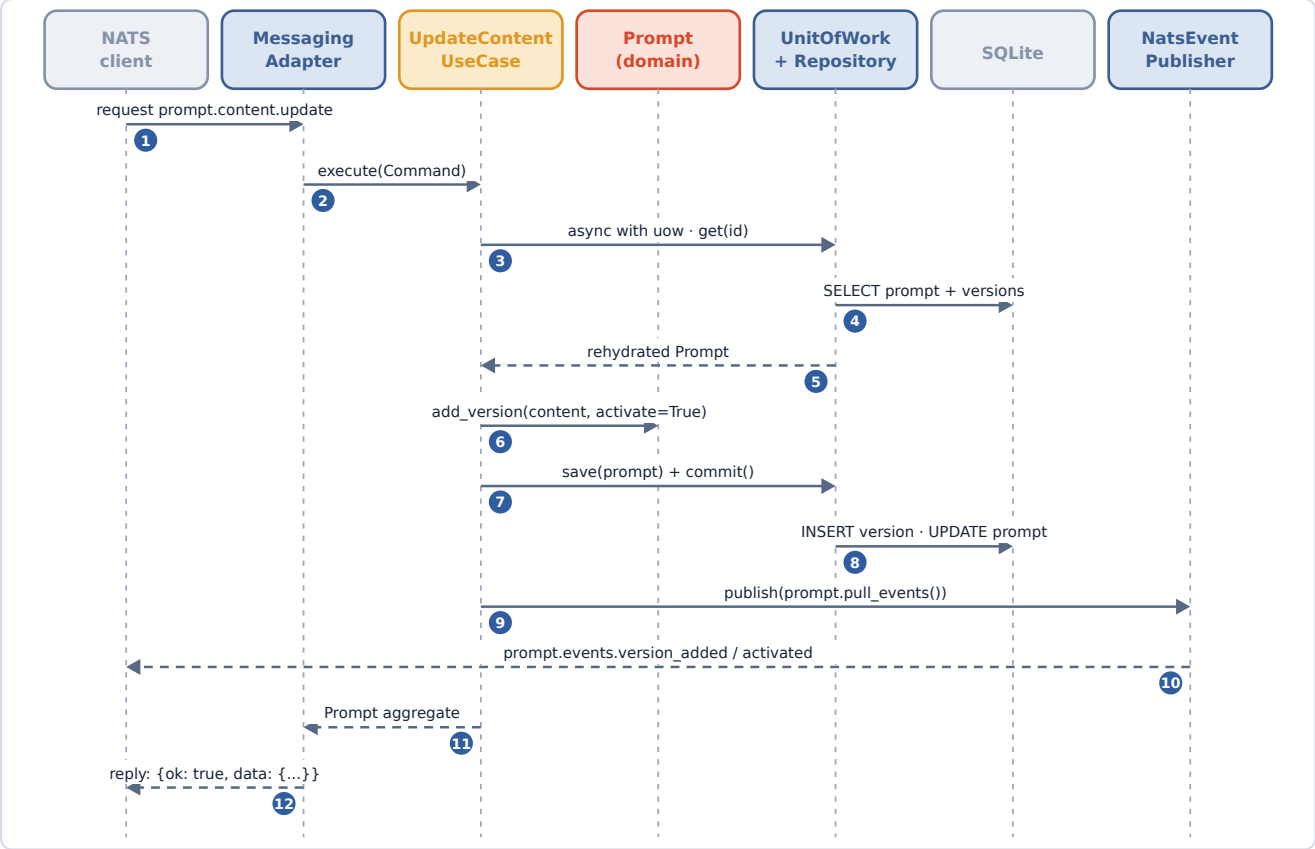


Figure 11 — The path of the message inside the core. Dashed lines are returns or publications; the numbers match the arrows.

#	What happens	Arrows	Who	Layer
1	The message arrives at the subject. The adapter decodes the JSON, finds the correlation id (from the header) and records it in every log.	1	`MessagingAdapter`	adapter
2	The handler validates only the format (UUID and text), builds the `UpdateContentCommand` and asks the factory for a new use case.	2	`handlers.py`	adapter
3	The use case opens the transaction (`async with uow`): the Unit of Work takes the write key and opens a session.	3	`UpdateContentUseCase`	app
4	The repository does the `SELECT` of the prompt and its versions; the mapper rebuilds the aggregate with `rehydrate`.	4, 5	`SqlAlchemyPromptRepository`	adapter
5	The use case calls `prompt.add_version(content, activate=True)`. The business rule runs here: it validates, numbers, activates and records two events.	6	`Prompt`	domain
6	The repository writes (`INSERT` of the new version, `UPDATE` of the prompt) and the Unit of Work commits. The key is released.	7, 8	`SqlAlchemyUnitOfWork`	adapter

#	What happens	Arrows	Who	Layer
7	Already outside the transaction, the use case hands the events (`pull_events()`) to the publisher, which sends them to NATS with the correlation id.	9, 10	`NatsEventPublisher`	adapter
8	The use case returns the prompt and the new version. The handler serializes them into the `PromptVersionResponse` shape, wraps it in the `ok` envelope and publishes it on the client's reply address.	11, 12	`MessagingAdapter`	adapter

9.2 The whole journey, from the browser

Now widen the camera. The same operation, started from the screen:

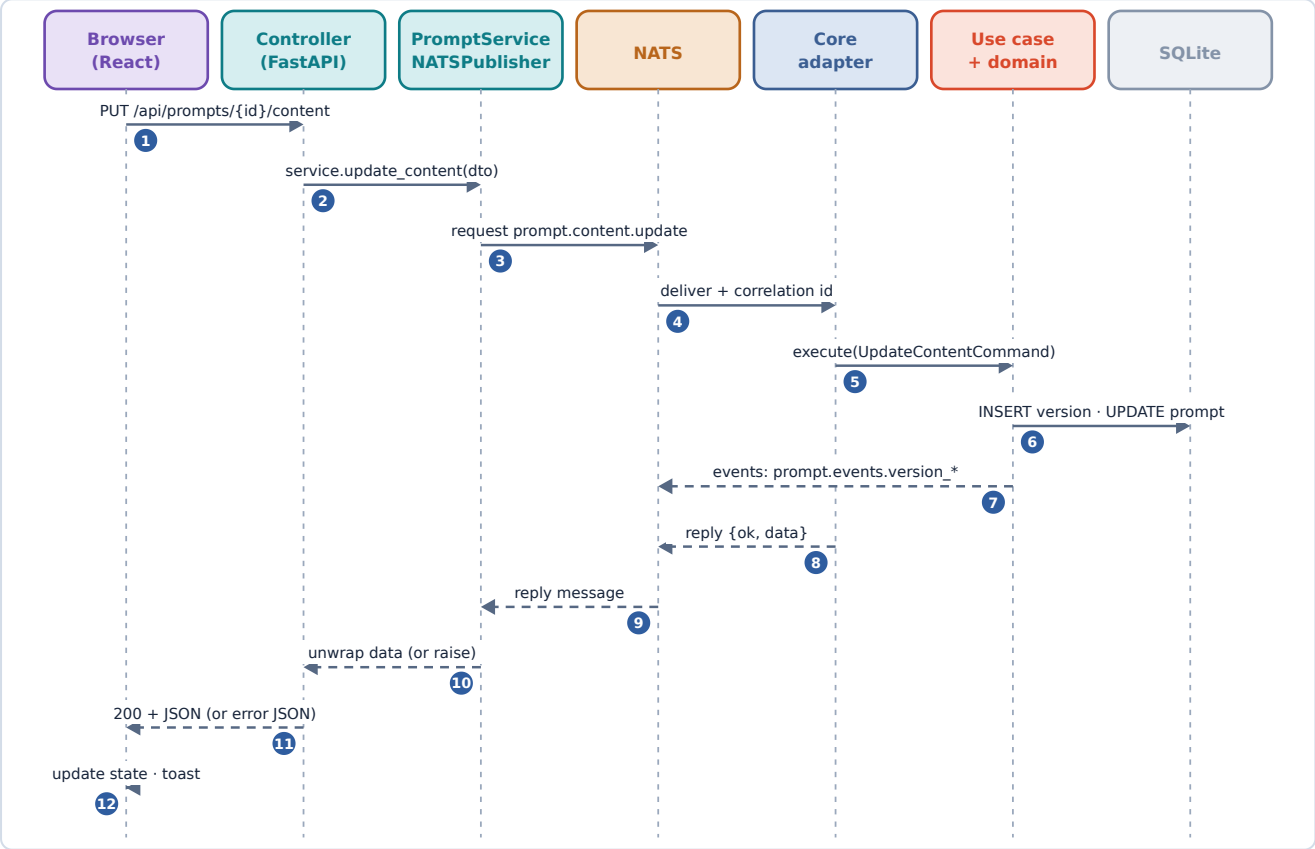


Figure 12 — The complete journey of one click, from the browser to SQLite and back.

1. **The click.** In `PromptDetailModal`, `save(true)` calls `updateContent(id, draft)`, and Axios sends `PUT /api/prompts/{id}/content` with the body `{"content": ...}`.
2. **The proxy.** In development the Vite server forwards it to `http://localhost:8000/prompts/{id}/content`.
3. **The controller.** FastAPI validates the path (a UUID) and the body (a Pydantic model), and calls `service.update_content(...)`.
4. **The service and publisher.** `PromptService` builds `{"prompt_id": "...", "content": "..."}`, picks the subject `prompt.content.update` and `NATSPublisher.request` sends it, waiting at most

three seconds.

5. **The core.** Everything in section 9.1 happens: use case, domain, SQLite, events, reply envelope.
6. **The reply.** NATS brings the envelope back. The publisher validates it, sees `ok: true` and returns only `data`. The controller returns that as the HTTP body, status 200.
7. **The screen.** The front end receives the new version, asks `reloadOne` to refresh that prompt (another `GET`, a full round trip through all the layers), closes the modal and shows the green toast "New version vN saved and activated".

9.3 What each observer sees

Who	What they see
The browser user	The modal closes, the card shows the new version badge and the new preview, and a toast confirms it.
The api logs	One request line, no error.
A subscriber of `prompt.events.>`	Two events, `version_added` and `version_activated`, both with the header `X-Correlation-Id`.
The database	A new row in `prompt_versions` and the `active_version_number` updated in `prompts`. Older versions stay intact.
The core logs	`event_received`, `use_case_started`, `prompt_saved`, `transaction_committed`, `event_published`, `use_case_completed`, `reply_sent`, all with the same `correlation_id`.

9.4 When something goes wrong

The same journey fails in different places, and every failure has a predictable ending:

What fails	Where it is noticed	What the user ends up seeing
The id in the URL is not a UUID	FastAPI, before any NATS message	`422 INVALID_REQUEST`, a toast with the message. The core never hears about it.
The core is not running	NATS: nobody subscribed to the subject	`503 CORE_UNAVAILABLE`.
The core is too slow	The api's three-second timeout	`504 CORE_TIMEOUT`.
The prompt is in the trash	The domain refuses (`PromptDeletedError`); the core replies `prompt_deleted`	`409 PROMPT_ALREADY_DELETED`; the screen keeps working.
The prompt does not exist	The use case (`PromptNotFoundError`); the core replies `not_found`	`404 PROMPT_NOT_FOUND`.
Something nobody planned	The central handler of either side	`500 INTERNAL_SERVER_ERROR`, with the details only in the logs.

CHAPTER SUMMARY

- A click crosses: component, service layer, HTTP, controller, service, publisher, NATS, messaging adapter, use case, domain, repository, SQLite, and the same path back.
- The correlation id follows the message through the core, from the reply to the events and logs.
- The lesson of architecture: NATS appears only in the messaging adapter and the api publisher; SQL only in the persistence adapter. The business rule (step 5 of 9.1) touches no technology. That is having the business at the center and the technology at the edges.
- Every kind of failure ends as the same standard JSON error, shown as a toast.

CHAPTER 10

10 Verifying that it works

Tests from top to bottom, and the lessons they gave

GUIDING QUESTION

How do we know that all of this really works, and does not merely look like it works?

10.1 Verify in layers

Every stage of the project was verified at the level where it made sense. The deeper the layer, the faster and more isolated the test; the further out, the closer to reality:

Level	What was tested	With what
Domain	Creation, versions, activation, life cycle, soft delete, events, rehydration, diff.	Pure Python, nothing external.
Application	The 11 use cases, including "nothing is saved nor published when the domain refuses".	Fake repository and publisher.
Persistence	Round trips to SQLite, time zones, rollback, 25 simultaneous commands, WAL mode, data after a restart.	Real SQLite in a temporary file.
The api	The publisher, the 11 service methods, the routes, the exceptions and handlers: 117 unit tests.	`pytest` with NATS and the service mocked. Nothing touches the network.
Core, end to end	Every subject, events, errors, timeout, clean shutdown.	Real NATS (Docker) and the real app.
Core smoke test	54 checks through the "front door", using only the NATS CLI.	`core/smoke_test.sh`
Api smoke test	48 checks over HTTP, with real NATS and a real core.	`api/smoke_test.sh`
Global smoke test	62 checks that the three pieces agree (see 10.3).	`smoke_test.sh` at the root
Front end	Types checked by the compiler; every screen exercised in a real browser against the real api.	`npm run build`, plus browser scripts

10.2 The smoke test

A **smoke test** comes from electronics: you turn the device on and see whether smoke comes out. In software, it is a quick check that the essentials work. The scripts are plain shell: they use only the NATS CLI or `curl`, exactly like a real user, send messages and compare the replies. Two small pieces hold the script up:

core/smoke_test.sh · send a request and compare results

```
req() {
  local subject=$1 payload=$2 cid=${3:-}
  local args=(request "$subject" "$payload" --timeout "$TIMEOUT" --raw)
  [[ -n "$cid" ]] && args+=(-H "X-Correlation-Id:$cid")
  RESP="$(nats "${args[@]}" 2>/dev/null)" || RESP="<no response within $TIMEOUT>"
}

expect_eq() { # <description> <expected> <actual>
  if [[ "$3" == "$2" ]]; then pass "$1"; else fail "$1" "expected: $2 | actual: $3"; fi
}
```

10.3 The global smoke test: do the three pieces agree?

The smoke test of each project checks that project against its own expectations. That is not enough: both sides could pass while disagreeing with each other. The global script (`smoke_test.sh` at the repository root) is built to catch exactly that:

1. It checks that NATS, the core and the api all answer, and tells you what to start if one does not.
2. It walks through every operation **over HTTP** and validates each JSON body against the api's own Pydantic models (`PromptResponse`, `PromptVersionResponse`, `PromptDetailResponse`, `VersionDiffResponse`, lists of them). If the core ever produced a field with a different name or type, this fails.
3. For each query, it asks the same thing **directly on NATS** and requires the HTTP body to be *identical* to the core's `data`. That proves the gateway only transports.
4. A prompt created over HTTP must be visible on NATS, and one created on NATS must be visible over HTTP.
5. Creating, changing, deleting and restoring over HTTP must publish the matching domain events (`prompt.events.created`, `version_added`, ...).
6. The core's errors must reach HTTP with the right status and code (404, 409, 422).

Result of the last run: **62 of 62** checks, 0 failures. The core's own smoke test passes 54 of 54, and the api's 48 of 48.

> **IN THE PROJECT · Contract tests, discovered by accident**
>
> The idea of validating real bodies against the api's models was not in the plan. It came from a finding: the first time the api documented its DTOs, the core still answered in an older shape, and nothing failed. Documentation and reality had silently drifted. The global smoke test is the guard against that drift.

10.4 Other verifications

- **Concurrency:** 40 simultaneous commands on the same prompt generated versions 2 to 41, with no loss and no duplicate.
- **Persistence and life cycle:** data survived a restart (the file in WAL mode) and `SIGTERM` stopped the app cleanly.
- **Timeout:** with the limit forced to 0.0001 s, the client received `timeout` and the log showed the formatted traceback.
- **Isolation:** no mention of infrastructure, adapters or libraries in `domain/`, `app/` and `port/`; `pyright` with 0 errors.
- **The browser:** compare, timeline, trash, activation and deletion were exercised with a real Chromium against the real api: success paths, simulated 500 and 409 errors, loading states, dark theme and a 375-pixel-wide screen.

10.5 Lessons from the way

What went wrong	What we learned
SQLAlchemy 2.1 asked for `greenlet` when used asynchronously.	Read the error message to the end: it said the extra to install (`sqlalchemy[asyncio]`).
The traceback came out as a raw dictionary in the log.	Two good libraries can conflict at the junction; test the error path, not only the happy one.
The smoke test failed intermittently with `not_found`.	Two instances of the app were on the same NATS and both answered (each with its own database). Without a queue group, every subscriber receives everything.
`restore` worked on a prompt that was not deleted.	The end-to-end test found a business-rule hole that reading the code did not show.
The api guessed the type of a core error by reading its English message.	Between systems, exchange *codes*, not prose. The core now sends one code per rule.
The api documented its DTOs but the core answered in another shape.	Documentation drifts silently. Add a test that compares the two (the global smoke test).
`openapi-typescript` crashed with the newest TypeScript.	A tool that uses a compiler's internal API is tied to its major version; pin it. (The project uses TypeScript 5.)
The split diff overlapped long lines.	`table-layout: fixed` plus no-wrap text overflows its cell. Give the table a minimum width and let the wrapper scroll.
Browser tests failed although the page was right.	Tests have bugs too: wait for what only the final content has, and match text exactly.

WATCH OUT · Scaling to several instances

Today, two instances of the core on the same NATS execute every command twice. To scale horizontally, the subscriptions need a *queue group* (NATS then delivers each message to only one member), and the shared database would need concurrency control beyond the local lock. It is a natural next step, not done yet.

CHECKPOINT

1. What does the global smoke test prove that each project's own smoke test cannot?
2. Why is it better for the core to send one error code per rule than a single generic one?
3. Why are the api unit tests allowed to mock NATS, and what do they *not* prove?

CHAPTER SUMMARY

- Each layer was tested at the right level; the api has 117 unit tests with NATS mocked.
- Three smoke tests exist: core (54), api (48) and global (62). The global one checks that the pieces agree.
- The best lessons came from tests that failed; they are worth thanking.

CHAPTER 11

11 Hands-on

Run, explore and extend the project on your own

> **GUIDING QUESTION**
>
> How do I use the system myself, and how do I extend it without breaking the architecture?

11.1 Bringing up the environment

You need Docker, `uv`, Python 3.13 and Node.js 22. Use one terminal per service:

Terminal

```
# 1) the NATS server (needs Docker)
cd core && docker compose up -d

# 2) the core (another terminal)
cd core && uv sync && uv run src/main.py

# 3) the api (another terminal)
cd api && uv sync && uv run uvicorn main:app --reload

# 4) the front end (another terminal)
cd front && npm install && npm run dev
```

Open `http://localhost:5173` for the application, and `http://localhost:8000/docs` for the api's Swagger page. Then run the full verification:

Terminal

```
./smoke_test.sh          # the global smoke test, from the repository root
```

11.2 Recipes

Replace `<ID>` with the `id` returned when you create a prompt. Open a second terminal running `nats sub 'prompt.events.>'` to see the events arriving while you work.

Straight to the core, with the NATS CLI

```
# create a prompt
nats request prompt.create '{"name":"greeting","content":"Hello\nworld"}' --raw
# new version (not active) and content change (creates and activates)
nats request prompt.version.create '{"prompt_id":"<ID>","content":"Hi\nworld"}' --raw
nats request prompt.content.update '{"prompt_id":"<ID>","content":"Hey\nworld\nend"}' --raw
# read, compare and go back
nats request prompt.get '{"prompt_id":"<ID>"}' --raw
nats request prompt.versions.compare '{"prompt_id":"<ID>","from_version":1,"to_version":3}' --raw
nats request prompt.version.activate '{"prompt_id":"<ID>","version_number":1}' --raw
# delete (soft), list and recover
nats request prompt.delete '{"prompt_id":"<ID>"}' --raw
nats request prompt.list.deleted '{}' --raw
nats request prompt.restore '{"prompt_id":"<ID>"}' --raw
# a command without a reply, with its own tracking code
nats pub prompt.delete '{"prompt_id":"<ID>"}' -H "X-Correlation-Id:my-trace"
```

The same operations over HTTP, with curl

```
curl -s -X POST localhost:8000/prompts -H 'Content-Type: application/json' \
     -d '{"name":"greeting","content":"Hello\nworld"}'
curl -s localhost:8000/prompts/<ID>
curl -s -X POST localhost:8000/prompts/<ID>/compare -H 'Content-Type: application/json' \
     -d '{"from_version":1,"to_version":2}'
curl -s -X DELETE localhost:8000/prompts/<ID>
curl -s localhost:8000/prompts/deleted
```

11.3 Exercises

#	Exercise	Hint
1	**Watch the events.** With `nats sub 'prompt.events.>'` open, create a prompt and change its content, first with the NATS CLI and then in the browser. Which events appear, in which order?	Look for the `X-Correlation-Id` header in the output.
2	**Read a diff.** Create a version 2 with one changed line, one removed and one added, then compare. Classify every entry.	Note how a "modified" line appears as removed + added in the contract.
3	**Cause every kind of error.** Invalid JSON, an unknown UUID, restoring an active prompt. What `code` comes back from the core, and what HTTP status from the api?	Compare Chapter 6 (codes) with Chapter 7 (statuses).
4	**Follow a correlation id.** Send a request with `-H "X-Correlation-Id:my-trace"` and find that code in the core's output.	`grep` or your terminal's search.
5	**Stop the core.** With the front end open, stop the core and click around. What does the user see, and why?	`503 CORE_UNAVAILABLE`, in a toast. Then start the core: does the page recover without a reload?

#	Exercise	Hint
6	**Challenge (domain).** Write a `DiffStrategy` that compares word by word.	Create a class in `src/domain/diff/` with `name` and `diff`, and swap the strategy in `main.py`. Which files did not need to change?
7	**Challenge (new feature).** "Rename prompt". Plan, layer by layer and project by project, what would change.	See the plan below, and count how many layers are touched.

IN THE PROJECT · Exercise 7: the map of the change

1. **domain:** a `Prompt.rename(name)` method (validates a non-blank name, updates the date) and a `PromptRenamed` event.
2. **port:** nothing changes; saving a modified prompt is the usual `save`.
3. **app:** `RenamePromptUseCase` and `RenamePromptCommand`, in the same mold as `DeletePromptUseCase`; it returns the prompt.
4. **adapter/messaging:** the subject `prompt.rename`, a handler and a `Route` in `build_routes`; the event gets its name in `_EVENT_NAMES`.
5. **adapter/persistence:** nothing changes; the mapper already copies the `name` field (`apply_state`).
6. **api:** a request DTO, a `PromptService.rename_prompt` method, a route (`PATCH /prompts/{id}`) and its tests.
7. **front:** a service function and an input in the details modal. Regenerate the types with `npm run gen:api`, and the compiler shows every place that must be updated.
8. **smoke tests:** new checks for the subject, the route and the event.

Notice: the database and the ports stay untouched. It is the architecture showing its value: the change touches only where it makes sense.

11.4 Natural next steps

Improvement	Why
Queue group in the subscriptions	Allow several instances of the core without executing each command twice.
Outbox pattern	Write the event in the same transaction as the data, so an event published after the commit can never be lost.
A read port (`PromptReader`)	Separate reading from writing in the ports and remove the read repository's concession.
Unit tests for the core	Turn the verification scripts into a permanent `pytest` suite that runs on every change.
Front-end tests	Add component tests and turn the browser scripts into a permanent suite.
Pagination, and the active version inside the list	Avoid returning every prompt at once, and spare the front end its request per card.

Improvement	Why
CORS in the api	Allow a production build of the front end served from another address.
AsyncAPI documentation	Formally describe subjects and payloads, so other teams can integrate.

APPENDIX A

A Glossary

The terms of this guide in plain language

Term	Meaning
Adapter	A piece that translates between the core and a concrete technology (NATS, SQLite).
Aggregate	A group of objects treated as a unit, with a root that protects the rules (Prompt + versions).
Asynchronous	Not standing still while waiting: while one task waits, another moves on (`async/await`).
Axios	The HTTP client library used by the front end.
Broker	The central server that receives and distributes messages (the NATS server).
Command	A use case that changes the state of the system.
Component (React)	A function that returns a piece of screen.
Composition root	The single place that creates and connects all the pieces (`main.py`).
Context manager	The `with` structure that guarantees something is opened and closed correctly.
Controller	In the api, the function bound to an HTTP route. It stays one line long.
CORS	The browser rule that decides whether a page may call an API at another address.
Correlation id	A tracking label that accompanies a message in replies, events and logs.
Dataclass	A class whose constructor and comparisons are generated automatically.
Dependency injection	Handing an object the pieces it needs, instead of letting it create them.
Diff	A comparison showing what changed between two texts.
Domain event	An immutable fact in the past that happened in the domain ("version activated").
Duck typing	"If it walks and quacks like a duck...": the shape of the object counts, not its inheritance.
DTO	A plain object that only carries data across a boundary.
Envelope	The standard reply format: `ok`, `correlation_id` and `data` or `error`.
Gateway	A thin entry point in front of a backend; here, the api.
Hexagonal	An architecture of "ports and adapters" that isolates the core from the technology.
Invariant	A statement that can never be false ("versions numbered with no gaps").
Lifespan	Code that runs when a FastAPI app starts and stops.
LLM	A large language model; it receives prompts and generates text.

Term	Meaning
Mapper	Code that translates between the domain model and the table model.
NATS	A messaging system based on subjects, with publish/subscribe and request/reply.
OpenAPI	A machine-readable description of an HTTP API; the source of Swagger and of the front end's types.
ORM	Maps database tables to classes of the language (SQLAlchemy).
Outbox	A pattern that records events together with the data so they are never lost.
Port	A contract (Protocol) that the core demands from the outside world.
Prompt	An instruction text sent to a language model.
Protocol	A description of the methods a class must have, without inheritance.
Proxy (Vite)	A development server feature that forwards `/api` calls to the real API.
Pydantic	A library that validates data against type annotations.
Query	A use case that only reads.
React	A library for building user interfaces from components.
Request / reply	A conversation in which the asker waits for the answer.
Rollback	Undoing everything a transaction did; also, activating an old version.
Soft delete	Deleting by "hiding": the data stays stored and is recoverable.
SPA	Single-page application: one page that redraws itself and talks to an API.
State (React)	Data a component remembers between draws.
Strategy	A pattern to swap an algorithm without touching whoever uses it.
Subject	The "topic" (channel) where NATS messages are published.
Timeout	The maximum waiting time before giving up.
Toast	A small temporary notification on the screen.
Transaction	A set of operations that is "all or nothing".
Type hint	An annotation of the expected type of a value (`str | None`).
Unit of Work	An object that manages a transaction and confirms or undoes everything.
uv	A tool that creates the Python environment and installs dependencies.
Vite	The development server and bundler of the front end.
WAL	The SQLite mode in which reading and writing coexist, via a "scratch page" of writes.

APPENDIX B

B File map

Where everything is, and which layer it belongs to

B.1 core/

File	Layer	Responsibility
`src/domain/entities/prompt.py`	domain	`Prompt`: the root aggregate, with every rule and event.
`src/domain/entities/prompt_version.py`	domain	`PromptVersion`: an immutable snapshot of the content.
`src/domain/status.py`	domain	`PromptStatus` and the table of permitted transitions.
`src/domain/events.py`	domain	The seven domain events.
`src/domain/exceptions.py`	domain	Business errors (`DomainError` and children).
`src/domain/diff/models.py`	domain	`ChangeType`, `DiffEntry` and `VersionDiff`.
`src/domain/diff/strategy.py`, `line_diff.py`	domain	`DiffStrategy` (contract) and `LineDiffStrategy` (line by line, with `difflib`).
`src/port/*.py`	port	The three contracts: `PromptRepository`, `EventPublisher`, `UnitOfWork`.
`src/app/commands/*.py`	app	Six write use cases; each returns the aggregate.
`src/app/queries/*.py`	app	Five read use cases (including `CompareVersions`).
`src/app/exceptions.py`, `_helpers.py`	app	`PromptNotFoundError` and `require_prompt`.
`src/adapter/persistence/models.py`	adapter	ORM tables and the UTC date type.
`src/adapter/persistence/mapper.py`	adapter	Domain to tables translation, both ways.
`src/adapter/persistence/prompt_repository.py`	adapter	The SQLAlchemy repository (write and read).
`src/adapter/persistence/unit_of_work.py`	adapter	The concrete transaction, with the write key.

File	Layer	Responsibility
`src/adapter/messaging/subjects.py`	adapter	The subject names.
`src/adapter/messaging/codec.py`	adapter	JSON to objects, field validation and envelopes.
`src/adapter/messaging/handlers.py`	adapter	Translation from subject to use case.
`src/adapter/messaging/serializers.py`	adapter	Domain objects to the JSON contract (`PromptResponse` and friends).
`src/adapter/messaging/server.py`	adapter	Subscriptions, timeout, replies and logs.
`src/adapter/messaging/event_publisher.py`	adapter	Publishes the domain events on NATS.
`src/adapter/messaging/correlation.py`	adapter	The correlation id per asynchronous task.
`src/adapter/messaging/errors.py`	adapter	Exception to error code.
`src/infra/settings.py`, `database.py`, `nats_client.py`, `logging.py`	infra	Environment configuration, SQLite engine, NATS connection, structlog + rich.
`src/main.py`, `docker-compose.yml`, `smoke_test.sh`, `pyproject.toml`, `uv.lock`	n/a	Composition root · local NATS · smoke test · dependencies.

B.2 api/

File	Responsibility
`main.py`	The FastAPI app, the NATS lifespan and the handler registration.
`controllers/prompt_controller.py`	The `/prompts` router: eleven one-line routes.
`services/prompt_service.py`	One method per use case: payload + subject + publisher.
`messaging/nats_publisher.py`	The only code that imports `nats-py`; maps core error codes to exceptions.
`models/prompt_models.py`	The request DTOs and the body-only DTOs.
`models/response_models.py`	`ApiWrapper[T]` and the response DTOs used for the documentation.
`utils/exceptions.py`	`BaseAppException` and its subclasses.
`utils/error_handlers.py`	The central handlers, and their registration.
`utils/json_utils.py`, `logger.py`	JSON encode/decode helpers and logging setup.
`tests/`	117 unit tests, organized by layer.
`smoke_test.sh`	48 checks over HTTP.

B.3 front/

File	Responsibility
`src/main.tsx`, `App.tsx`	Startup, the state of the list, the hash router, the navigation bar and the toasts.
`src/api/client.ts`	The shared Axios instance and the error interceptor.
`src/api/prompts.ts`	One function per api endpoint.
`src/api/schema.d.ts`	Types generated from `openapi.json` (never edited by hand).
`src/types.ts`	Short aliases for the generated types.
`src/components/PromptList.tsx`, `PromptCard.tsx`	The searchable list and its cards.
`src/components/CreatePromptModal.tsx`, `PromptDetailModal.tsx`	Creation, and details with copy, edit and delete.
`src/components/ComparePage.tsx`, `VersionSelector.tsx`, `DiffViewer.tsx`	Version comparison.
`src/components/TimelinePage.tsx`, `VersionTimeline.tsx`	The version history and its quick actions.
`src/components/TrashPage.tsx`, `ConfirmModal.tsx`, `Modal.tsx`	The trash, and the dialogs.
`src/components/Toaster.tsx`, `ThemeToggle.tsx`, `GeometricBackground.tsx`, `FloatingAddButton.tsx`	Notifications, theme, decoration and the "+" button.
`vite.config.ts`, `openapi.json`, `.env.example`	Dev server and proxy · the api contract snapshot · configuration.

B.4 repository root

File	Responsibility
`smoke_test.sh`	The global smoke test: HTTP, NATS, core, contract and cross-checks.
`docs/`	This guide (`prompt-vault-study-guide.pdf`), the interactive map (`prompt-vault-map.html`) and the guide's source (`docs/guide/`).

APPENDIX C

C Going further

Reading and documentation to continue where this guide stopped

C.1 Books and articles

Work	Why read it
Hexagonal Architecture, Alistair Cockburn (article, 2005)	The origin of the idea of ports and adapters, by the author.
Domain-Driven Design, Eric Evans (2003)	The source of aggregates, entities, events and the "rich model".
Architecture Patterns with Python, Harry Percival and Bob Gregory (2020)	Repository, Unit of Work, events and messaging, all in Python. Very close to this project.
Patterns of Enterprise Application Architecture, Martin Fowler (2002)	The classic description of Repository and Unit of Work.
Designing Data-Intensive Applications, Martin Kleppmann (2017)	Messaging, consistency and why exactly-once delivery is hard.
Thinking in React (react.dev)	How to break a screen into components and where state should live.

C.2 Official documentation

Technology	Where to look
Python (asyncio, dataclasses, typing)	docs.python.org
uv	docs.astral.sh/uv
NATS	docs.nats.io
SQLAlchemy 2.x (ORM and asyncio)	docs.sqlalchemy.org
FastAPI and Pydantic	fastapi.tiangolo.com · docs.pydantic.dev
pytest	docs.pytest.org
structlog and rich	structlog.org · rich.readthedocs.io
React and TypeScript	react.dev · typescriptlang.org
Vite and Tailwind CSS	vite.dev · tailwindcss.com
OpenAPI	spec.openapis.org

CONCEPT · How to study from here

Pick a folder (start with `domain/`), read the files from top to bottom, and for each function ask: "who calls it? what does it protect?" Then change something small, run a smoke test and see what happens. Code is learned by reading, running and breaking things on purpose.

APPENDIX D

D Answers to the checkpoints

The questions of each chapter, answered

Ch.	Question	Answer
2	What is `uv.lock` for?	It records the exact versions of everything installed, to reproduce the same environment on any machine. `uv` generates it; editing by hand risks breaking that guarantee.
2	What is `int | None`?	A value that can be an integer or nothing (`None`). In `active_version_number`, `None` means "no active version".
2	What if there is an error before `commit()` in an `async with uow`?	On leaving the block, `__aexit__` does a rollback: nothing done in the transaction stays stored.
3	Why does `domain` not import `sqlalchemy`?	To stay independent from technology: it can be tested without a database, and the rules survive a change of technology.
3	What to change to swap SQLite for PostgreSQL?	`adapter/persistence` and the configuration in `infra/database.py` (URL and driver). `domain`, `app` and `port` stay intact.
3	Port versus adapter?	A port is the contract (a Protocol); an adapter is a concrete implementation of that contract, tied to a technology.
4	Why is `PromptVersion` frozen?	Because history is immutable: a version, once created, never changes. Switching the active version only moves the pointer.
4	What does `restore()` do on an active prompt?	It raises `PromptNotDeletedError`: only what is deleted can be restored.
4	A word-by-word diff?	A new class with `name` and `diff(old, new)` (a `DiffStrategy`), handed to the use case in `main.py`.
5	Command versus query?	A command changes the state (uses a transaction and publishes events); a query only reads (no transaction, publishes nothing).
5	Why publish after the commit?	Not to announce to the world something that, because of a failure while saving, did not happen.
5	Why does the use case receive the `uow` in the constructor?	Dependency injection: the use case does not know how the transaction is made, which allows swapping it (even for a fake, in tests).
6	What is the correlation id for?	To trace a message from beginning to end: it shows up in the reply, in the events and in every log line.
6	Why a new instance of each use case per message?	Each one carries its Unit of Work and its database session, which cannot be shared by simultaneous messages.

Ch.	Question	Answer
6	What does WAL improve?	Reading and writing coexist: readers are not blocked while someone writes.
7	Why does `NATSPublisher.request` return only `data`?	The gateway transports and does not reshape. The envelope (`ok`, `correlation_id`) is protocol; the controller's job is to return the content.
7	Why do controllers have no `try / except`?	Error handling is central: handlers registered on the app turn every exception into the same JSON format, so no route can forget or differ.
7	What can make the api answer 503 or 504 with no bug in the code?	The core is stopped (no responders, 503), publishing to NATS fails (503) or the core is slower than the three-second timeout (504).
8	Why call `/api/...` and not `localhost:8000` directly?	The api has no CORS, so the browser would block the cross-origin call. The Vite proxy makes the front end and the api look like one origin.
8	What does the stale response guard solve?	Two requests in flight where the older answers last: without the guard the screen would show a diff that does not match the selected versions.
8	Why does the front end not compute the diff?	The algorithm is a business rule and lives in the core. Computing it again would duplicate logic and risk two different answers.
10	What does the global smoke test prove that the others cannot?	That the pieces agree with each other: the HTTP bodies match the api's models and the core's `data`, and both sides see the same state.
10	Why one error code per rule?	Between systems a code is a stable contract; matching English prose breaks silently when someone rewords a message.
10	Why may api unit tests mock NATS, and what do they not prove?	Mocks isolate the api's own logic and make the tests fast and deterministic. They do not prove that the real core answers as the mock does; that is what the smoke tests are for.